

Swim Meet Data Model Design

https://gemini.google.com/app/1373e9763bd9d3c5

User prompt: i'm still struggling with this decision... implement timer screen as a form (hidden inputs for startTimestamp, stopTimestamp, timeMs, athleteId, isExhibition)... maybe incrementally submitting (ie hitting "Start" button actually submits any fields that have been populated, so server gets updates throughout and can reflect in near-real time on admin panel)... should those form posts go through clientAction, which then could figure out how to dispatch? or, should button pushes directly invoke websocket RPCs (eg startWatch(startTimestamp)? or, should it be a mix - RPC calls for optimistic/real time signals (watch started, swimmer name updated) directly to RPC and form POST to action/clientAction for submit? trying to balance complexity/readability and simplicity, with slight offline tolerance, vs sticking to established patterns

Response: The tension you're feeling is between **transient telemetry** (which is ephemeral and real-time) and **business-critical state transitions** (which demand durable guarantees and offline recovery).

Trying to force both through an incremental HTML form submission, or vice-versa forcing both through ad-hoc WebSocket RPCs, is why the design feels awkward.

The Clean Separation: Transient Telemetry vs. Durable Record

Think of what happens on a lane timer’s screen across the lifecycle of a single heat:

Interaction	Nature	If a packet is lost / offline	Best Transport
Timer taps "Start"	Ephemeral Telemetry ("Lane 3 watch is ticking")	Doesn't matter. The race is still running. The official time is determined by the stop touch, not the live tick.	WebSocket Message (Fire-and-Forget)
Swimmer walk-up edited	Minor Context	Nice to see live on the admin desk, but not official until results lock.	WebSocket Message (Fire-and-Forget)
Timer taps "Finish" / "Submit"	Durable Business Record	Catastrophic. If this time is lost, the athlete has no record of their swim.	Form POST (clientAction + Cookie Outbox)

Why "Incremental Form Posts" Fall Down on Mobile

Submitting an HTML form (even via clientAction) every time someone taps "Start" or updates a field introduces subtle friction on mobile:

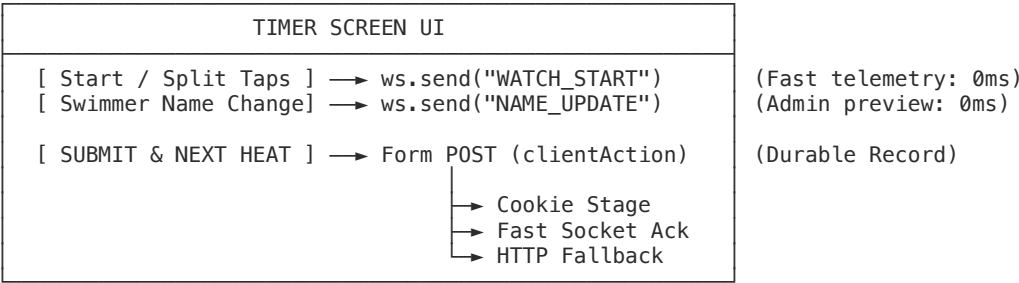
1. **Remix Fetcher Collisions:** If a timer taps "Start" and then 1.5 seconds later changes a radio button, you either abort the first in-flight fetcher or juggle multiple concurrent fetchers (fetcherStart.submit(), fetcherEdit.submit(), fetcherFinish.submit()).

2. **Cookie Churn:** Writing and clearing disaster-recovery cookies on intermediate ticks like "watch started" clutters your fallback outbox with state that doesn't actually need disaster recovery.

3. **Admin Noise:** The admin doesn't need transactional database commits for "Timer 3 clicked start." The admin just needs an in-memory pulse on their grid showing that Lane 3's timer is awake and running.

The Architecture: Ephemeral Sockets + Form Actions

Keep the screen modeled around **one standard Remix <Form> / <fetcher.Form>**, and use raw WebSocket emissions purely as **instant telemetry**.



1. Ephemeral Signals (Direct Socket RPC)

These are 1-line calls that bypass Remix action pipelines entirely. If the phone is offline behind a bulkhead, the socket is dead, and the message drops silently with zero consequences:

```
// Inside TouchStopwatch component:
const onStart = () => {
  // Fire-and-forget telemetry to admin grid
  ws?.send(JSON.stringify({
    type: "TIMER_STARTED",
    heat: currentHeat,
    lane: currentLane,
```

```

    startedAt: Date.now()
  }));
};

```

The Admin DO receives this in memory and broadcasts to the admin screen, turning Lane 3's box from gray to a pulsing yellow clock. No database writes, no cookies, no Remix revalidations.

2. The Submission (Standard Remix Form & `clientAction`)

When the race ends, the volunteer reviews the numbers and taps **"Submit & Advance"**. This is the **only** operation that touches `clientAction`:

```

<Form method="post" onSubmit={handleFormAdvance}>
  <input type="hidden" name="timeMs" value={stopwatchMs ?? ""} />
  <input type="hidden" name="partnerWatch1" value={watch2} />
  <input type="hidden" name="partnerWatch2" value={watch3} />
  <input type="hidden" name="swimmerOverride" value={nameOverride} />
  <input type="hidden" name="isExhibition" value={String(isExhibition)} />
  <input type="hidden" name="isDq" value={String(isDq)} />

  <button type="submit" disabled={!canSubmit}>
    Submit & Next Heat
  </button>
</Form>

```

3. What `clientAction` Does

Inside `clientAction`, you have your unified dispatch logic:

1. **Stage the cookie:** Pack the form fields into `pending_time_${lane}`.
2. **Try the Fast Socket:** If `ws.readyState === WebSocket.OPEN`, send the final submission over the socket and wait up to 300ms for an ACK.
3. **HTTP Fallback:** If socket is closed or ACKs time out, call `serverAction()`.
4. **Advance:** Return immediately so the UI moves forward to the next heat without waiting on network roundtrips.

Why This Is the Sweet Spot

- **Clean Mental Model:** Form = "Record of Result". WebSocket = "Live Deck Telemetry".
- **Simple Components:** No array of competing fetchers or tangled state machines. Your form contains standard inputs, and your start/stop triggers are just lightweight button callbacks.
- **Resilient Where It Counts:** If the Wi-Fi drops, the admin misses the yellow "pulsing clock" animation for that heat (which is harmless), but the athlete's 28.42s final time remains safe in the cookie and syncs the moment connectivity returns.

User prompt: is it worth trying the socket for form submit? or, just ditch the dual approach and always submit via POST? also, one point: i've implemented using all POSTs today, and it's pretty responsive...not 5-15ms, but a couple hundred, and there's no contention...your recommendation makes sense and is definitely better and not too difficult to implement, so i'm thinking of changing, but i'm torn because of the old "if it ain't broke..." adage.

Response: **Stick with pure HTTP POST.** Do not build the dual-path socket submission.

If you already have POST submissions working cleanly, measuring a couple hundred milliseconds, and running contention-free, adding a socket write-path is classic premature optimization.

The Reality of "15ms vs. 200ms" on Deck

In a video game or a collaborative Figma canvas, the difference between 15ms and 200ms is huge.

In a swim meet:

1. **The touch itself is already recorded locally:** Your digitizer touch listener captures the finish timestamp down to the millisecond the instant the thumb hits the glass. That number is frozen.
2. **Post-race pacing is slow:** Swimmers don't leave the water instantly. The starter waits 10 to 30 seconds before sounding the horn for the next heat.
3. **The UX difference is invisible:** Whether an ingestion payload hits the database in 20ms or 200ms, the timer is already looking at their partner's backup watch, and the admin at the computer table can't perceive a 180ms delta anyway.

What You Avoid by Ditching the Dual Path

Introducing the WebSocket as an alternate write path brings hidden edge-case complexity:

- **Split-Brain Logic:** You now have two code paths that ingest, parse, validate, and acknowledge the exact same business record. Any change to validation rules must be mirrored in both.
- **Race-Condition Deduplication:** If the socket attempt experiences a slight stall, drops to HTTP fallback, and both packets eventually hit the DO within milliseconds, your DO must flawlessly deduplicate without dropping or misattributing times.
- **ACK Timeout Tuning:** If you set an ACK timeout to 300ms, but deck LTE has a transient 350ms latency spike, the client will constantly fire duplicate HTTP fallbacks for messages that actually succeeded over the socket.
- **Maintenance Debt:** You trade a rock-solid, boring HTTP pipeline for a custom request-ACK-retry protocol layered over raw WebSockets.

The Optimal Division of Responsibility

Leave your mutation layer alone and let each protocol do what it does best:

Mutation Flow (Durable State)

Timer Client → Standard HTTP POST (Remix Action) → DO / SQLite → Clears Cookie

Event Flow (Ephemeral & Live Reads)

DO Broadcasts → WebSocket → Admin Grid / Live Clocks / UI Revalidation

- **HTTP POST** owns writes: It gives you atomic requests, cookie lifecycle integration, native browser retry mechanisms, and zero custom socket plumbing.
- **WebSockets** own reads/telemetry: They push real-time broadcasts to the admin desk, broadcast heat advances, and ping live clocks.

"If it ain't broke, don't fix it" applies directly here. Keep your working POST pipeline—it's simpler, easier to debug in browser dev tools, and more than fast enough for the deck.

User prompt: so, confirming your recommendation - still stick to WS for ephemeral signals, but just use the HTTP POST for the final form submission?

Response: Yes, exactly. That is the cleanest, most resilient split:

- **WebSocket (Upstream & Downstream) for Ephemeral Signals:**
 - Timer → Server: "WATCH_STARTED", "WATCH_STOPPED", live runner pulse.
 - Server → Clients: "HEAT_ADVANCED", live clock sync, live admin grid updates, `revalidator.revalidate()`.
 - *Failure Mode:* If a packet drops or the phone goes offline for 10 seconds, nobody cares. The race is still happening, and missing a yellow pulse on the admin screen changes nothing.
- **HTTP POST for the Final Form Submission:**
 - Timer → Server: The actual record of truth (final elapsed time, partner watches, DQ/exhibition flags).
 - *Failure Mode:* Guarded by your cookie/queue outbox, standard HTTP response codes (200 OK), and clean server-side cookie clearance. It handles intermittent connectivity natively without building a custom ACK/retry protocol over raw sockets.

You keep the real-time responsiveness where it adds visual value, and you keep boring, reliable HTTP where data loss is unacceptable.

User prompt: ok - love this ideal. WS/DO for ephemeral messages, form data submits via HTTP POST for anything critical that needs to be persisted (and would still leverage the payload-as-a-cookie approach to ensure durability). obviously, for timer messages, watch start/stop are WS messages... and submitting a time(s) for an event/heat/lane is an HTTP POST. what about swimmer assignment? today my timer UI allows for updating the assigned swimmer for the current event/heat/lane... timers can be selected from the list of all known athletes in the meet, or specify a name/team/gender for "walk up" swims. should this be its own separate form submission? or an ephemeral WS message? or just sent with the final payload for the event (event/heat/lane + athlete + stopwatch results)

Response: Bundle it directly into the **final HTTP POST submission**, but emit an **ephemeral WebSocket notification on selection** as a live deck preview.

This gives you the best of both worlds: zero-friction forms for the timer, instant situational awareness for the admin, and zero transactional clutter in the database.

Why It Belongs in the Final HTTP POST

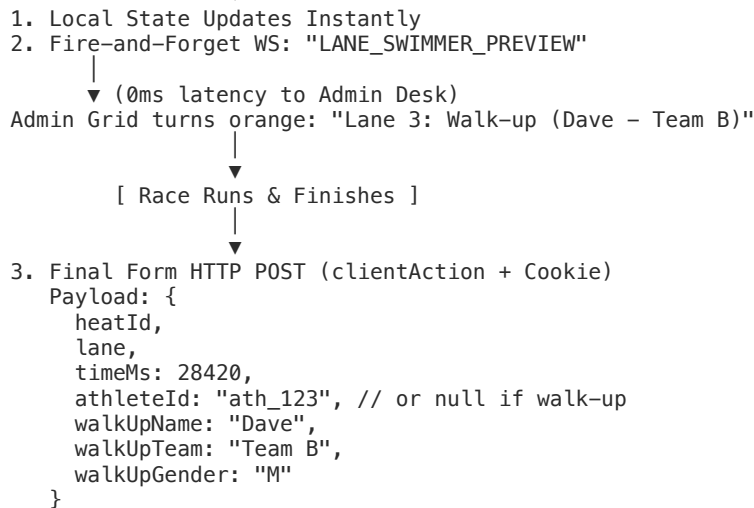
If you make swimmer assignment its own dedicated form submission:

1. **Network Overhead & Race Conditions:** If a timer selects "Sarah M.", then 3 seconds later realizes it's actually "Dave T.", you fire two independent mutation cycles and cookie updates before a race has even started.
2. **Scratches & Abandoned Heats:** If a timer selects a walk-up swimmer, but the swimmer then gets pulled from the blocks or the heat is reseeded, your database now has an orphaned deck assignment that needs an explicit undo/scratch workflow.

3. **Atomic Heat Record:** An unofficial swim is fundamentally a single tuple: (**Who swam, In what lane, What was their time**). Keeping those values together in one final payload ensures the lane assignment and the recorded time commit as an atomic unit.

The Hybrid Flow: Ephemeral WS Preview + Atomic Final Commit

Timer selects "Sarah Miller" (or types walk-up "Dave – Team B")



How Each Side Experiences This:

- **The Timer:** Completely seamless. They pick the swimmer or type a name, and those values simply sit in hidden inputs (`<input type="hidden" name="athleteId" ... />`). When they hit "Submit & Advance", everything commits together.
- **The Admin Desk:** When the timer picks the name, the WebSocket broadcast immediately gives the admin a visual heads-up on their live grid: "Lane 3 changed swimmer to Dave (Team B)". If the admin knows that's wrong, they can immediately flag it or wave to the timer before the race even starts.
- **The Database / DO SQLite:** Only writes once, at the conclusion of the swim. No intermediate table churn or cleanup jobs needed.

Edge Case: What if the Timer Doesn't Know Who Is Swimming?

Often behind the blocks, a kid steps up right at the whistle and the timer doesn't have time to search the dropdown or type a name.

Because the payload handles this at final submission:

- The timer can leave the assignment as whatever was pre-seeded (or blank) and simply hit **"Submit Time"**.
- The admin receives the time on their grid flagged as an **Unverified / Open Lane**.
- The admin—who usually has the scratch sheets and official coach lineup cards right at the table—can assign the athlete from the admin console during post-heat review.

Bundling the swimmer assignment into the final POST keeps the timer's mental model dead simple: **set up your lane, time the race, and tap one button to commit**.

User prompt: only thing to consider is that, currently, in my app, updating the name on a lane also updates my "entries" table, so a coach can in real time or later see that swimmer was entered in that event... maybe that's something that should change, or maybe it impacts the decision here about assignment being part of the lane result HTTP POST?

Response: This touches on a core business-logic rule: **When does deck reality become an official meet entry?**

In competitive swimming—especially developmental and high school meets—an athlete who walks up to an empty lane hasn't just been "renamed" on a timer sheet; they have effectively been entered into that event on the fly (which affects athlete event limits, scoring eligibility, and coach psych sheets).

Writing to your `entries` table immediately on dropdown select creates three distinct problems on deck:

The Risks of Eager Entry Creation

1. The "Oops, Wrong Kid" Problem:

Timer in Lane 3 selects "Johnny Smith". Three seconds later, they realize it's actually "Johnny Davis" and switch it. You now have an orphaned entry for Johnny Smith in the 100 Free that either clutters the database or requires an immediate deletion/scratch cascade.

2. The "Ghost Swim" / False Start Problem:

A coach sends a swimmer over to Lane 4 as an exhibition swim. The timer selects them. Before the horn goes off, the starter calls a false start or the swimmer is pulled from the blocks. If the entry was committed eagerly, that swimmer is now officially recorded as entered in that event—potentially putting them over their allowable individual event entry limit (e.g., 2 individual events max).

3. Network Drops on Deck:

If the timer's phone drops connection right as they select the dropdown, an eager POST fails or sits in a pending state before the timer has even touched the stopwatch.

The Clean Solution: Commit the Entry in the Final POST Transaction

Keep swimmer assignment inside the **final HTTP POST**, but update your backend transaction so that creating or linking the Entry happens atomically alongside the result.

Timer submits Final Form HTTP POST

```

↓
[ Cloudflare D0 / SQLite Transaction ]
1. Does an Entry already exist for this (athlete_id, event_id)?
   ↳ YES: Link existing entry_id to this lane_slot.
   ↳ NO (Walk-Up):
       a. Create a new `Entry` record with status="deck_entered".
       b. Link newly created entry_id to this lane_slot.
2. Insert TimerSubmission / Lane Result.
3. Broadcast to Admin & Coaches: "ENTRY_CREATED_AND_TIMED".

```

Why This Works Better:

- **True State of Record:** An entry created on deck is only valid if the swimmer actually steps up and swims the race. Committing the entry when the final time is posted ensures you only create official entries for swims that genuinely occurred.
- **Live Visibility Remains Intact:** By emitting the ephemeral WebSocket "LANE_SWIMMER_PREVIEW" when the timer touches the dropdown, the coach and admin can still see in real time: "Lane 3: Walk-up (Dave)". It just doesn't become a permanent, scored database entry until the race touches the wall.
- **Coach Lineup Reconciliation:** When the admin hits "Approve Heat", the coach's view immediately shows the new entry under that event with its official posted time.

What About Pre-Seeded Swims (Scratches & Swaps)?

When a timer changes the name on a lane that already had a seeded swimmer, you're handling a **swap**:

- Swimmer A was pre-seeded in Heat 2, Lane 3.
- Swimmer B actually walks up and swims it.

In your final POST handler:

1. Mark Swimmer A's entry as `scratched` or `no_show` (so historical seed records aren't lost).
2. Create or assign Swimmer B's entry to that `lane_slot`.
3. Save the time.

Keeping all of that in the final HTTP POST handler isolates your business logic to a single, transactional endpoint, prevents ghost entries in your database, and keeps the timer screen fast and uncomplicated.

User prompt: below is my current design. what do you think? also... i'm really debating with whether different entities (entry [aka seed] vs result) are even needed, or if everything manifests as a "swim" (coach enters a swimmer which creates a swim (we'd add seed time to the record, the "swims" are reseeded, etc) entry meet_id event_id athlete_id seed_time? heat? lane? swim meet_id event_id heat lane athlete_id athlete_name athlete_team watches[] startTimestamp stopTimestamp timeMs status? (OK, DQ) official_time decided_at add/update/delete of an entry in an event triggers an automatic re-seed of that event. there should probably be some limit on when entries are allowed to be updated until (eg once an event has been marked closed and we're onto a later event, entry changes/reseeding is moot?) should entries be updated to reflect reality of what was swum? if yes - the trigger should be the admin accepting the swim (so, making it official that the given athlete swam the given event). if no - this might make sense so there's a record of original plan (entries) vs actual swims (results)? timers never update entries. WS/DO broadcast messages are used to update athlete, set exhibition status, and report watch start/stop. this makes incremental updates to the swim record, which is broadcast out to others. it is OK this only works intermittently if internet is flaky (it's informational only). HTTP POST form submissions (using cookie-as-payload) are used to submit timers official "time sheets" - event/heat/lane + swimmer + watches. thus, we have a natural hierarchy of resilience: - WS/DO... fastest and most preferred, but least reliable - HTTP POST... still fast (if connected), resilient via cookies, but still just semi-reliable (device failure or never regaining internet means a swim is not recorded) - paper lanesheet ... still kept as backup, simply ignored if everything goes well, but good to have in case all the digital stuff fails

Response: Collapsing everything into a single Swim entity is tempting because it feels like less code, but doing so will break standard meet operations the moment deck reality diverges from the psych sheet.

Keeping Entry and Swim (or Result) decoupled is the right call, and your hierarchy of resilience is practical. Here is an architectural critique of the design, addressing the entity debate, deck entry limits, and state lifecycles.

1. The Core Dilemma: Entry vs. Swim (Why Collapsing Them Breaks Down)

If you collapse everything into a single **Swim** record, an entity must simultaneously represent **Coach Intent** (seed times, pre-meet psych sheets, team event rosters) and **Deck Execution** (physical lane slot, actual swimmer, recorded watches, DQ status).

That creates four major failure modes:

- **Scratches & Empty Lanes:**

If Sarah is seeded in Heat 2, Lane 4, and scratches 5 minutes before the race, what happens to the record? If you delete the **Swim**, you lose the record that she was ever entered (and potentially entered into too many events). If you keep the **Swim** and mark it **scratched**, you now have a record tied to a physical heat and lane that has no actual swim data.

- **Deck Swaps (The "Ghost Seed" Problem):**

If Johnny is pre-seeded in Lane 3 with a 24.50 seed time, but Marcus walks up to swim exhibition in that lane instead, mutating that single record overwrites Johnny's seed time and psych sheet history.

- **Relay Lineups:**

Relay entries exist long before the coach decides which 4 swimmers step onto the blocks. A relay **Entry** has a seed time; the eventual **Swim** has four specific athlete IDs and split times.

- **The "Plan vs. Reality" Audit Trail:**

Coaches and scorers frequently ask at the end of a meet: *"Did we scratch him, or did he No-Show?"* Keeping **Entry** intact preserves the original meet plan.

The Clean Entity Relationship

```
[Entry] (Intent: athlete_id, event_id, seed_time, status: declared | scratched)
  |
  |< (optional 1:1 link)
[Swim] (Execution: event_id, heat, lane, athlete_id_actual, watches[], official_time, status: OK | DQ | DNS)
```

- An **Entry** can exist without a **Swim** (a scratched swimmer).
- A **Swim** can exist without a pre-existing **Entry** (a walk-up exhibition swimmer added on deck).
- When an admin approves an official heat, the system simply links the **Swim** to the matching **Entry** if one exists.

2. When Should Re-seeding Be Locked?

"there should probably be some limit on when entries are allowed to be updated until..."

Your intuition is correct. Auto-reseeding cannot be allowed once deck timing begins.

If a coach scratches an athlete from Event 4 while Event 3 is in the water, auto-reseeding Event 4 would scramble heat and lane assignments across the entire pool while timers are already holding paper backup sheets and swimmers are lining up behind the blocks.

The Rule:

1. **Meet State: Seeded** → Adding/deleting/scratching an entry automatically recalculates heats and lane seeds.
2. **Meet State: In Progress / Event State: Locked** → Auto-reseeding is completely frozen.
 - Scratches do **not** reseed the event; they simply mark the lane as an empty lane / scratch.
 - Deck additions are assigned manually by the admin to an open lane in an existing heat (or placed into an added "exhibition heat" at the end).

3. Should Entries Be Updated to Reflect Reality?

"should entries be updated to reflect reality of what was swum? if yes - the trigger should be the admin accepting the swim..."

No, do not mutate the Entry to mirror the Swim. Keep them distinct and reconcile them on Admin Approval.

Here is why:

- **The Audit Trail:** The **Entry** is what the coach submitted on Tuesday (e.g., entered with a 1:04.20). The **Swim** is what happened on Saturday (e.g., swum in Lane 2 with a 1:03.85, or scratched).
- **Walk-Up Swims:** When a timer submits a walk-up swimmer, the admin accepting the heat can optionally create a **deck_entry** record with **seed_time: null (NT)**.

- **Final Results Output:** When generating the meet results export, you query the approved Swim records, not the original entries.

4. Refining the Resilience Hierarchy

Your proposed 3-tier hierarchy is sound and maps cleanly to deck realities:

Tier 1: WebSockets / DO (Fastest, Ephemeral)

- ↳ Watch Start/Stop pulses, live swimmer previews.
- ↳ Loss tolerance: 100%. If dropped, meet operations continue unaffected.

Tier 2: HTTP POST + Cookie-as-Payload (Durable Record)

- ↳ Full heat submission (heat, lane, athlete, raw watches, status).
- ↳ Loss tolerance: 0%. Protected by device storage, auto-retry on reconnect.

Tier 3: Paper Lane Sheet (Physical Air-Gap)

- ↳ Written backup in the timer's clipboard pocket.
- ↳ Solves catastrophic hardware failures (dropped phone in pool, dead battery).

5. Recommended Schema Tweaks

Given the decoupled model, here are minor schema adjustments to make your database and DO SQLite storage clean and queryable:

Entry (Intent)

```
entry
  id: uuid (primary key)
  meet_id: string
  event_id: string
  athlete_id: string
  seed_time_ms: integer? (null = NT)
  status: 'declared' | 'scratched' | 'deck_added'
  created_at: timestamp
```

Swim (Deck Reality)

```
swim
  id: uuid (primary key)
  meet_id: string
  event_id: string
  heat_number: integer
  lane_number: integer

  entry_id: uuid? (null if unseeded walk-up)
  athlete_id: string? (populated on assignment)
  athlete_name_override: string? (walk-up name)
  is_exhibition: boolean

  watches: json [
    { device_id: string, start_epoch: number, stop_epoch: number, elapsed_ms: number }
  ]

  official_time_ms: integer?
  status: 'pending' | 'official' | 'dq' | 'dns'
  dq_reason: string?
  decided_at: timestamp?
  decided_by_user_id: string?
```

Summary Evaluation

- **Keep Entry and Swim separate:** It preserves pre-meet seed integrity, cleanly models scratches and walk-ups, and prevents deck chaos.
- **Freeze reseeding:** Auto-reseed during pre-meet prep; lock heat/lane assignments once deck staging starts.
- **Keep your resilience hierarchy:** Ephemeral WS for live deck pulses, HTTP POST + cookie queue for official times, and paper lane sheets for the ultimate air-gap backup.

User prompt: all makes sense, except i don't follow the proposed schema tweaks - heat/lane were removed from "entry"... so, would "seeding" mean creating "swim" records for each entry and assigning heat/lane to them?

Response: Yes, exactly. **"Seeding" is the act of materializing Swim records from Entry records.**

When you decouple them this way:

1. Before Seeding (Meet Setup):

- Coaches submit **Entry** records: "Sarah is entered in the 50 Free with a 28.40 seed time."
- There are no heats, no lanes, and no **Swim** records yet.

2. The Seeding Step (Generating the Meet Program):

- An algorithm or admin seeds the event (e.g., standard circle seeding or slowest-to-fastest).
- Seeding **creates the empty pool grid as Swim records** for each heat and lane, linking each one to its corresponding `entry_id`.
- For example, Heat 2, Lane 4 gets a **Swim** row created with `entry_id`: "entry_sarah_123".

3. During the Meet (Deck Operations):

- Timers and admins interact **only with the Swim records**.
- Watches, touches, DQs, and official times are written directly to that **Swim** row.
- If Sarah scratches on deck, the **Swim** row simply gets marked `status`: 'dns' (Did Not Swim), or another athlete's ID is swapped onto that **Swim** row. The original **Entry** record remains untouched in historical declared status.

Why Keeping Heat/Lane on Entry Causes Problems

If you keep heat and lane on the **Entry** table directly, you run into three deck traps:

- Empty Lanes Have No Identity:** If an 8-lane pool has an empty Lane 1 and Lane 8 in Heat 1, there is no **Entry** for those lanes. If a walk-up swimmer steps into Lane 8 at the last second, the timer's app has no pre-existing entity in the database to attach watch times to unless they create an artificial "Entry" on the fly.
- Deck Swaps Overwrite Seeds:** If Swimmer B takes Swimmer A's lane, updating **Entry** wipes out who was originally seeded there and scrambles historical psych sheets.
- Scratches Leave Dead Holes:** If you delete or nullify the heat/lane on an **Entry** when a kid scratches, queries that render the heat sheet (`SELECT * FROM entries WHERE heat = 2`) suddenly lose track of the empty lane entirely.

The Two Ways to Model It

Depending on whether you prefer fewer tables or strict physical normalization:

Option A: Pre-Generated Swim Rows (Simplest & Most Common)

When an event is seeded, you pre-populate all physical slots as **Swim** rows:

- 6 heats × 8 lanes = **48 Swim rows** created upfront.
- Seeded lanes have `entry_id` and `athlete_id` pre-filled.
- Unseeded / empty lanes have `entry_id`: null and `status`: 'open'.
- Timers are always writing to an existing row (`WHERE event_id = ? AND heat_number = ? AND lane_number = ?`).

Option B: The Pure Physical Model (LaneSlot + Swim)

If you want to be pedantic about relational design:

- LaneSlot** represents physical pool space (`event_id`, `heat`, `lane`, `entry_id`).
- Swim / Result** represents the recorded performance data attached to that slot.

For most club, high school, and developmental apps, **Option A is the sweet spot**: seeding creates the **Swim** records with their heat/lane assignments, and deck timing simply fills in the blanks on those existing rows.

User prompt: so, "seeding" creates swim records for each entry record, assigning heat/lane. if/when entries are added/removed and a re-seed happens, would be just regenerate heat/lane assignments (per the rule above that changes to entries after the meet has started won't auto re-seed)?

Response: Yes, that is the exact pattern: **pre-meet reseeding simply deletes and regenerates the unswum swim records for that event.**

Because `entry` holds coach intent (who is entered and their seed time) and `swim` holds physical assignment, pre-meet reseeding is completely non-destructive to athlete entries.

How the Reseeding Lifecycle Works

1. Pre-Meet Phase (`event.status === 'seeded'`)

When a coach scratches an athlete or adds an entry 30 minutes before the meet starts:

1. Update, insert, or mark entry.`status = 'scratched'`.
2. Delete all existing `swim` rows for that event:
`DELETE FROM swims WHERE event_id = ? AND status = 'pending';`
3. Run your seeding algorithm (e.g., standard spearhead lane assignment: 4-5-3-6-2-7-1-8 across heats).
4. Re-insert the fresh `swim` rows with new `heat_number`, `lane_number`, and `entry_id` references.
5. Broadcast "EVENT_RESEDED" over the WebSocket so any open screens update their heat sheets.

2. Live Deck Phase (`event.status === 'locked' or 'in_progress'`)

Once the meet starts and an event is marked locked (or timers have checked into heat 1):

- **Automatic reseeding is completely disabled.**
- Adding or scratching an entry no longer touches the global heat sheet.
- **Scratches:** The existing `swim` row remains at Heat 2, Lane 4; its status simply changes to `dns` (or `scratch`), leaving the lane empty in the water.
- **Deck Adds / Walk-ups:** The admin manually assigns the athlete to an existing open lane's `swim` row, or appends a new exhibition heat (e.g., Heat 7) at the very end of the event without scrambling Heats 1 through 6.

This keeps pre-meet entry management fast and flexible, while completely protecting timers and swimmers from last-second heat scrambles on deck.

User prompt: hmm... the way i was thinking about this, and why entry had heat/lane, was that "seeding" would impact only entries... every change to entry would trigger re-seed (simply updating heat/lane for every entry in that event) up until whatever re-seed deadline we set. a "swim" is only ever generated by a timer... the timer's UI would know about all entries, so would pre-populate the "swim" form based on entry if available (if not available and a swimmer is in the lane, the timer would manually enter, ie a "walk up")... no need for scratches/no shows in this model. where does it break down?

Response: The model of **Entry holds heat/lane and Swim is only born when a timer submits** is conceptually neat because it feels like pure event sourcing: *Plan sits in Entry, Reality is recorded in Swim*.

For a basic dual or developmental meet where everything goes smoothly, this works. However, it breaks down around six specific pool-deck edge cases:

1. The "Ghost Scratch" / Unaccounted Lane Problem

If a swimmer is seeded in Heat 2, Lane 4, steps up to the blocks, but walks away before the start (or simply doesn't show up):

- Under your model: The lane is empty. The timer doesn't time anyone and doesn't hit "Submit". Therefore, **no Swim record is ever generated**.
- **Where it breaks:**
 - To the computer, that heat now looks **incomplete / hung**. Did the timer forget to press submit? Did the phone drop offline? Or was the lane truly a no-show?
 - The admin reconciliation grid can't easily distinguish between a **dropped network submission** and an **empty lane** without an explicit `Swim` record stating `status: 'dns'` (Did Not Swim).
 - In standard meet scoring and high school eligibility, an unexcused No-Show (NS) often disqualifies the athlete from the rest of the meet. If there's no record generated, that audit trail is missing.

2. Multi-Timer Crowdsourcing Breaks the 1:1 Identity

If you run **two or three timers on Lane 4** (the crowdsourced model you established earlier for high school/developmental timing):

- Under your model: What does a timer submission create?
 - If Timer A taps submit, does it create the `Swim`?
 - When Timer B taps submit 2 seconds later for the same heat/lane, what do they update? They can't just "create a swim"—they have to find the `Swim` Timer A created, append their watch to `watches[]`, or write to an intermediate table.
- If `Swim` already exists (even in a pending state) for that physical slot, both Timer A and Timer B are simply appending raw submissions to a known target slot.

3. The Re-Seed Deadline Race Condition

If `heat` and `lane` live directly on `Entry`, and reseeding mutates those fields on `Entry`:

- Consider what happens 2 minutes before the deadline when Event 1 is already in the water, and a coach scratches a kid from Event 2.
- The system reseeds Event 2: every `Entry` in Event 2 has its `heat` and `lane` updated in the database.
- **The breakdown:**
 - Timers on deck are already standing behind their lanes holding **printed heat sheets** or looking at a cached offline screen.
 - If the database silently shuffles swimmers into different lanes or heats right before the start, the timer's screen and the physical kid standing in front of them suddenly disagree.
 - Because `Entry` was mutated in place, you've overwritten the state that matched the paper clipboard.

4. Swaps & Walk-Ups Create Dual-Identity Clashes

Suppose Sarah was seeded into Heat 1, Lane 3 via `Entry`. On deck, Sarah feels sick, and the coach sends Marcus to swim exhibition in that lane instead.

- The timer changes the name to "Marcus" and submits the `Swim`.
- **The breakdown:**
 - What is the state of Sarah's `Entry`? Her entry still says `heat: 1, lane: 3`.
 - If you query "Who swum in Heat 1, Lane 3?", `Entry` says Sarah, but `Swim` says Marcus.
 - If Sarah later swims in Heat 3, or if you run a report on whether Sarah competed, you have conflicting records in your database: Sarah has an `Entry` placing her in Lane 3, but Marcus has the `Swim`.

5. Official Time Calculation vs. Raw Ingestion

If `Swim` is generated directly by the timer:

- Who calculates `official_time`?
- If the timer's phone submits `timeMs: 28420`, and that creates the `Swim`:
 - What happens if the admin reviews the backup watches and overrides the time to 28.65?
 - What if a stroke & turn judge disqualifies the swimmer 60 seconds *after* the race finishes?
 - If `Swim` is just "the timer's form post," it entangles the **raw watch capture** with the **official scored result**.

How to Keep Your Simpler Mental Model Without the Bugs

If you really want to avoid pre-generating empty `Swim` rows for every lane, you can still use your model with **two small safeguards**:

1. Require Timers to Submit "Empty Lane / DNS":

If no one is in the lane, the timer must still hit a "No Swimmer / Empty" button that submits a `Swim` with `status: 'empty' or 'dns'`. This lets the DO and the admin know: *"Lane 3 has checked in and reported empty; don't wait for a watch time."*

2. Key `Swim` Strictly by (`event_id`, `heat`, `lane`):

Treat `Swim` as the **Physical Result of that Lane**, not the athlete.

- `Entry = (meet_id, event_id, athlete_id, seed_time, assigned_heat, assigned_lane)`
- `Swim = (meet_id, event_id, heat, lane, actual_athlete_id, official_time, status)`

3. Hard Lock on Reseeding:

The moment the admin marks an event as `locked` (or the first heat of that event steps onto the blocks), permanently disable the auto-reseed trigger. Any late changes after that point must be handled purely as deck adjustments in `Swim`, leaving `Entry` static.

User prompt: problem (1) above is definitely legit (and, the safeguard (1) above - submit DNS signal equally helpful, except for the no internet case). note that swims are already keyed by event id/heat/lane, and swim is the only official record of results, so 2+3+4 aren't problems. 5 isn't an issue - the admin always sets official times by accepting timers submissions or entering one manually. this is also probably only used for dual meets (or, triple dual)... realistically, anyone running an invitational is going to be using Hy-Tek meet manager. i'm coming around to the "heat and lane on swim record only" idea, but want to ensure all context is considered.

Response: High school dual and double-dual meets have specific deck quirks that make the "heat and lane live on `Swim` only" pattern even more advantageous than in big invitationals.

Evaluating this through the lens of high school dual meet reality clarifies several key operational dynamics:

1. The Dual Meet Reality: Lane Allocations vs. Psych Sheet Seeding

In invitationals, seeding is mathematical (circle seeding, slowest-to-fastest based purely on entry times).

In dual meets, pool geography is political:

- **Odd vs. Even Lanes:** Home team gets Lanes 1, 3, 5; Visiting team gets Lanes 2, 4, 6 (or 2-3-6 vs. 4-5-7 in an 8-lane pool).
- **Lineup Adjustments:** Coaches are playing a live chess game. If a coach sees the other team's lineup for the 100 Back, they frequently shuffle their varsity point-scorers right before the deadline.
- **Exhibition Heats:** Heats 2 and 3 of the 50 Free are often free-for-all JV or exhibition heats thrown together on deck so every swimmer gets a race.

When heat and lane are divorced from entry:

- **Entry is just the coach's declaration:** *"We declare Sarah and Megan for the 100 Fly; seed times are 1:04.10 and 1:08.50."*
- **Swim is the slot in the pool:** The dual-meet template stamps out the physical grid:
 - Heat 1 (Varsity Scoring): Lane 1 (Home), Lane 2 (Visitor), Lane 3 (Home), Lane 4 (Visitor), etc.
 - Heat 2 (JV / Exhibition): Open slots.

If a coach pulls an athlete or swaps two swimmers between Lane 1 and Lane 3 at the last minute, **you don't re-seed the event**. You simply point the two Swim records at the updated athlete_ids. The underlying declared Entry remains untouched.

2. Solving the "No Internet" DNS Safeguard

The primary hurdle identified with on-the-fly swim generation was:

If a kid no-shows and the lane is empty, how does the admin know the difference between an empty lane and a dropped network submission?

When Swim records are materialized ahead of time for the heat:

- An empty lane is a row that already exists in your Meet Durable Object's SQLite table (status: 'open' or 'unclaimed').
- **Online path:** The volunteer taps "No Swimmer / Empty Lane", emitting a quick DNS signal.
- **Offline path (timer has no signal):** The lane remains in pending on the admin grid. When the race touches the wall and the admin looks up at the pool, they visually see Lane 5 was empty.
- Because the Swim row already exists at (event_id, heat_number, lane_number), the admin can simply tap **"Mark DNS / Empty"** directly on their console with one click.
- When the timer's phone eventually reconnects, any queued empty payload matches the admin's resolution and merges cleanly.

3. Dual-Meet Scoring Eligibility & Exhibition Rules

High school dual meets enforce strict participation rules:

- Maximum 4 events per swimmer (max 2 individual).
- Scoring limits (e.g., in a 6-lane pool, only top 3 places score; max 2 scorers per team in an individual event).
- Exhibition designation: Coaches can enter non-scoring swimmers, but their times still count for individual season PRs or section/state qualifying cuts.

Keeping Entry separate from Swim makes compliance trivial:

- **The Rule Check:** Query swims WHERE athlete_id = ? AND status != 'dns' to verify an athlete hasn't exceeded their daily event quota.
- **Exhibition Status (is_exhibition):** This is a property of the Swim, not necessarily the athlete. A varsity swimmer might swim the 200 Free for points, but swim the 100 Breast as exhibition in Heat 2. Storing is_exhibition on the Swim row keeps scoring rules cleanly decoupled from the athlete's master profile.

4. What the Dual-Meet Lifecycle Looks Like

1. Pre-Meet: Coaches Declare Lineups
 - └ Creates `Entry` rows (athlete_id, event_id, seed_time)
2. Meet Initialization: Generate Grid
 - └ Generates `Swim` rows for all scheduled heats/lanes based on team lane agreements:


```
(event_1, heat_1, lane_1, home_swimmer_a)
(event_1, heat_1, lane_2, visitor_swimmer_b)
(event_1, heat_1, lane_3, home_swimmer_c)...
(event_1, heat_2, lane_1, null) [Open Exhibition Slot]
```

3. Deck Execution: Timers & Admin Run Heats
 - └ Timers hit stopwatch and submit form to /heats/:heat/lanes/:lane
 - └ Updates the existing `Swim` record with:
 - raw watches[]
 - walk-up athlete override (if lane was open or swapped)
 - └ Admin clicks "Approve Heat":
 - Computes `official\_time` (median/average)
 - Sets `status: 'official'`
 - Calculates dual-meet team points (e.g., 6-4-3-2-1-0)

Summary Checklist: Why This Model Holds Up

1. **Psych sheet safety:** Pre-meet seed times never get destroyed by deck swaps.
2. **Deterministic Admin UI:** The admin grid always knows exactly how many lanes exist per heat—it doesn't have to wait for timers to connect to discover which lanes were swum.
3. **Frictionless Walk-Ups:** Open/JV heats simply show up as empty slots waiting for a timer or admin to tag a name to them.
4. **Dual-Meet Native:** Supports odd/even team lane assignments without fighting mathematical circle-seeding algorithms.

User prompt: one smaller question - should a swim record really need to reference an entry record?

Response: No, a Swim record does **not** strictly need a foreign key reference to an Entry record.

In fact, making entry_id required—or even heavily relying on it—adds unnecessary relational friction. At most, it should be an **optional, nullable convenience link**, and you can omit it entirely if you want a simpler architecture.

Why Direct Entry References Break Down on Deck

If a Swim depends on an Entry, every swim that happens on the pool deck must be reconciled back to pre-meet planning paperwork. On a real pool deck, that expectation falls apart quickly:

1. Walk-Up / Exhibition Swimmers:

In JV heats or open lanes, a coach tells an extra swimmer, "Go hop into Lane 5 for the 50 Free." The kid steps onto the block. If Swim requires an entry_id, you cannot record that race until someone first fabricates an Entry record.

2. Last-Second Lane Swaps:

A coach swaps the swimmer in Lane 2 with the swimmer in Lane 4 right as the whistle blows. If Swim is tied to entry_id, you must either re-point foreign keys across rows or mutate the original entries. If Swim simply references athlete_id, updating the lane assignment is a single field change.

3. Relay Changes:

Coaches regularly change relay legs mid-meet based on who has energy left. The relay entry might have been declared generically under the school name; the actual Swim records the four kids who swam the race.

What a Swim Actually Needs

A Swim is a physical, historical event. It only needs to know:

- **Where and when did it happen?** (meet_id, event_id, heat_number, lane_number)
- **Who swam it?** athlete_id (plus snapshot fields like athlete_name or team_id for quick display/walk-ups)
- **What was the performance?** watches[], official_time_ms, status (OK, DQ, DNS), and is_exhibition

```
swim
  id: uuid
  meet_id: string
  event_id: string
  heat_number: integer
  lane_number: integer
  athlete_id: string?      <-- Direct entity reference
  athlete_name_override: string? <-- Quick fallback for unlisted walk-ups
  team_id: string?
  is_exhibition: boolean
  watches: json
  official_time_ms: integer?
  status: 'pending' | 'official' | 'dq' | 'dns'
```

How Entry and Swim Relate Without a Direct FK

Instead of a hard database link, the relationship is **natural and derived**:

- **Pre-Meet:** A coach declares an entry: (athlete_id: "sarah_1", event_id: "ev_50_free", seed_time: 28.50).
- **Seeding:** The meet initialization places "sarah_1" into (event_id: "ev_50_free", heat: 1, lane: 4).
- **Deck Results & Reporting:** When you want to compare seed vs. actual time in results sheets, you join on the composite key:

```
SELECT
  s.heat_number,
  s.lane_number,
  s.athlete_id,
  e.seed_time_ms,
  s.official_time_ms
FROM swims s
LEFT JOIN entries e
  ON e.meet_id = s.meet_id
  AND e.event_id = s.event_id
  AND e.athlete_id = s.athlete_id;
```

If Sarah swam the race, the query pulls her seed time. If a walk-up swimmer hopped in, e.seed_time_ms returns NULL (rendered as "NT" for No Time) without any orphaned foreign keys or failed constraints.

The Verdict

- **Do not make entry_id mandatory.**
- If you include an entry_id column, keep it purely as a nullable shortcut for pre-seeded swims.
- Better yet: **Omit entry_id from Swim altogether.** Point Swim directly to athlete_id. It keeps deck timing completely autonomous, makes lane swaps trivial, and eliminates foreign key constraint failures when deck reality diverges from the psych sheet.

User prompt: should "is_exhibition" be a column on entry, swim, or both? and, on swim - should it be it's own column, or one of the statuses?

Response: is_exhibition should be on **both** entities, and on Swim, it belongs as its **own dedicated boolean column**, not as a status.

Here is why:

1. Why is_exhibition Must Live on Both Entities

• On Entry (Pre-Meet Intent):

Coaches declare exhibition status during pre-meet lineup submissions. In high school dual meets, varsity scoring entries are capped (e.g., maximum 3 scoring swimmers per team per event). Any extra swimmers entered in that event must be declared as exhibition ("X") upfront so the scoring system and opposing coach know they cannot earn team points.

• On Swim (Deck Reality):

Deck adjustments happen constantly. A coach might enter an athlete as a scoring varsity swimmer, but after an earlier event causes an entry limit crunch or a strategic shift, they tell the table: "*Lane 2 is swimming exhibition.*" Conversely, a walk-up swimmer placed into an open lane in a JV heat is almost always marked exhibition on the fly.

- **The Rule:** When seeding or creating the Swim, copy the default from Entry.is_exhibition. But allowing Swim.is_exhibition to be toggled independently ensures deck reality can change without rewriting the original declared entry history.

2. Why It Should NOT Be a status on Swim

Status represents **operational and technical validity**; exhibition represents **scoring eligibility**. Conflating them creates an orthogonal state problem.

Consider the lifecycle of an exhibition swim:

1. **Before the race:** It is pending.
2. **During the race:** The watches are recorded and the time is calculated.
3. **Official verification:** It is approved and becomes an official time (official).
4. **Disqualification:** What happens if an exhibition swimmer false starts or performs an illegal butterfly kick?
 - If status = 'exhibition', how do you record that they were **disqualified**?
 - Does their status become 'dq'? If so, you've lost the record that they were exhibition.
 - If their status stays 'exhibition', how does the admin know their time was thrown out?

By keeping status orthogonal to is_exhibition, states remain distinct and clear:

Performance Outcome	status	is_exhibition	Scored for Team Points?	Eligible for Season PR / State Cut?
Clean Varsity Swim	'official'	false	Yes	Yes
Clean Exhibition Swim	'official'	true	No	Yes
Illegal Kick on Exhibition	'dq'	true	No	No
Exhibition No-Show	'dns'	true	No	No

Recommended Schema Design

```

entry
  id: uuid
  meet_id: string
  event_id: string
  athlete_id: string
  seed_time_ms: integer?
  is_exhibition: boolean DEFAULT false <-- Declared intent

swim
  id: uuid
  meet_id: string
  event_id: string
  heat_number: integer
  lane_number: integer
  athlete_id: string?

  is_exhibition: boolean DEFAULT false <-- Scoring eligibility

  status: 'pending' | 'official' | 'dq' | 'dns' <-- Operational validity
  official_time_ms: integer?
  dq_reason: string?

```

This ensures an exhibition swimmer who swims a personal best or achieves a state qualifying time retains an official, recognized time record, while your scoring engine simply filters on WHERE status = 'official' AND is_exhibition = false when tallying team points.

User prompt: one more - should 'watches' be its own table, like: watch id swim_id startTimestamp stopTimestamp timeMs submitted_at submitted_by vs having it as a json field on the swim?

Response: watches should be a **separate relational table** rather than a JSON blob on swim.

While JSON columns are convenient for unstructured data, timing records in a swim meet have concurrency and audit requirements that JSON handles poorly.

1. Concurrent Multi-Timer Ingestion (No Race Conditions)

If you have 2 or 3 volunteers timing Lane 4:

- **With a JSON column:** When Timer A and Timer B tap "Submit" at roughly the same second, both send their watch times to the DO. If watches are stored in a JSON array, you must either read the JSON, append, and rewrite the array, or use JSON patch functions. Under concurrent requests, Timer B can easily overwrite Timer A's watch, requiring locking or merge conflict resolution.
- **With a separate watch table:** Each timer submission is a simple, atomic INSERT:

```

INSERT INTO watches (id, swim_id, start_timestamp, stop_timestamp, time_ms, submitted_by)
VALUES (?, ?, ?, ?, ?, ?);

```

Two or three timers can submit simultaneously without locking, race conditions, or risking array clobbering.

2. Built-In Idempotency & Deduplication

Network hiccups mean retry envelopes and background flushes will inevitably send duplicate payloads.

With a separate table, deduplication is handled natively by SQLite using a unique constraint on (swim_id, submitted_by) or a dedicated idempotency_key:

```

CREATE TABLE watches (
  id TEXT PRIMARY KEY,
  swim_id TEXT NOT NULL REFERENCES swims(id) ON DELETE CASCADE,
  device_id TEXT NOT NULL,

```

```

start_timestamp INTEGER,
stop_timestamp INTEGER,
time_ms INTEGER NOT NULL,
submitted_at INTEGER NOT NULL,
idempotency_key TEXT UNIQUE
);

```

If a retry fires twice, INSERT OR IGNORE drops the second packet without parsing a JSON document or checking for duplicate array elements.

3. Transparent Audit Trail for Disputes

When a coach challenges a result at the scorer's table ("Why was my swimmer given a 28.40 instead of a 28.15?"):

- A normalized table preserves the full provenance: exact touch timestamps, which device submitted it, and when the submission arrived.
- The admin UI can easily query and display all raw inputs:

```
SELECT * FROM watches WHERE swim_id = ? ORDER BY time_ms ASC;
```

- If a timer's watch started late or was demonstrably faulty (e.g., a 1.5-second outlier), the admin can flag or delete that single watch row without mutating the parent swim record.

4. Official Time Calculations in Pure SQL

Because SQLite in Cloudflare Durable Objects is fast and embedded, computing official times from a separate table is straightforward:

- **Single watch:** That watch becomes the official time.
- **Two watches:** Average the two.
- **Three watches:** Take the middle value (median).
- **Outlier detection:** Check `MAX(time_ms) - MIN(time_ms) > 300` in the query itself.

```

SELECT
  COUNT(*) as watch_count,
  MIN(time_ms) as min_time,
  MAX(time_ms) as max_time,
  AVG(time_ms) as avg_time
FROM watches
WHERE swim_id = ?;

```

When Would JSON Win?

A JSON column makes sense only if:

1. One person enters all 1–3 watch times into a single form on one device ("Digital Clipboard" mode).
2. You never need to query, filter, or index individual watch times.

Because crowdsourced timing with multiple devices is a core goal, **a separate watch table is substantially cleaner, safer for concurrency, and easier to debug.**

User prompt: ok - next implementation detail... am i best off using react Context/context provider to allow my views (eg admin, timer, coach) to be able to access the latest meet data (including updates as WS messages arrive ... idea is they'd update the context)?

Response: **No, do not use React Context to manage meet state.**

Introducing a React Context to hold real-time meet data will fight directly against Remix's architecture and introduce serious performance pitfalls on mobile devices.

Why React Context Breaks Down in This Architecture

1. Excessive Re-Renders Under WebSocket Activity

React Context triggers a re-render across **every single consumer component** whenever any value inside the context changes.

- If a timer in Lane 1 submits a watch time, the WebSocket broadcasts an update.
- If that update updates a top-level MeetContext:
 - The Admin screen re-renders (expected).
 - The Coach split view re-renders (unnecessary).
 - **Every single Timer phone currently running an active stopwatch re-renders.**

- On a low-end phone in direct sunlight, triggering React component re-renders while a volunteer is running a 60fps stopwatch causes dropped frames, touch latency spikes, and battery drain.

2. You Duplicate Remix's Built-in State Engine

Remix already provides a built-in, route-scoped reactive data store: **Route Loaders + useRevalidator()**.

- If you put meet state into a React Context, you create two competing sources of truth: the data from `useLoaderData()` and the data from `useMeetContext()`.
- Every component then needs custom logic: *"Should I read from loader data, or do I check if context has a newer socket payload?"*

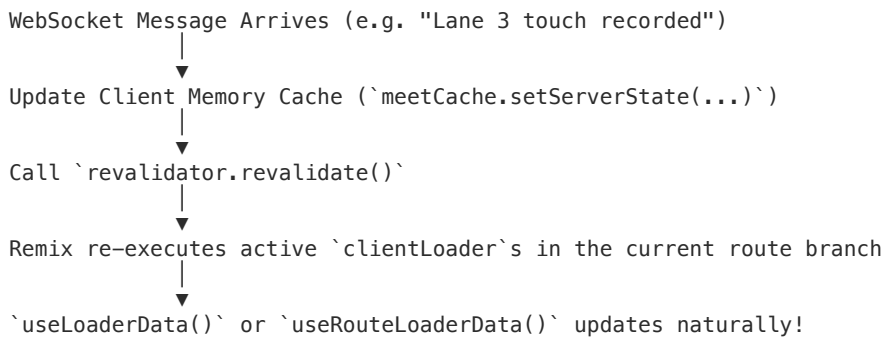
3. Hydration Mismatches & Double Paints

When a page loads via SSR:

1. The server renders HTML using `loader()` data.
2. The browser mounts, renders the HTML, and then initializes the `MeetContext` provider (often with an empty or transitional default state).
3. If context takes over state management, it immediately triggers a secondary client-side render cascade.

The Idiomatic Remix Solution: Local Cache + useRevalidator()

Instead of React Context, keep state updates strictly external to the React tree, using the pattern established earlier:



Why This Is Superior to React Context:

1. **Targeted Re-Renders:** Remix only re-renders the specific route components whose loader data actually changed. A timer in Lane 6 won't render when Lane 1 submits.
2. **Zero Custom Hooks:** Any screen that needs the latest data simply calls:


```
const { meet } = useRouteLoaderData<typeof rootLoader>("routes/meets.$id");
```

 No `useContext()`, no provider wrappers, and no prop drilling.
3. **Automatic Offline Alignment:** When `clientLoader` runs during revalidation, it pulls straight from `meetCache`. If the phone is offline, it reads local storage seamlessly without needing a Context reducer to handle offline states.

When /s React Context Useful Here?

React Context is great for **static, non-frequently changing configuration** that doesn't trigger continuous re-render storms:

- **Timer Identity:** Providing the current volunteer's device ID and selected lane assignment (`{ laneNumber: 3, deviceId: "dev_xyz" }`).
- **Active Mode / Permissions:** Providing `{ role: "admin" }` or `{ highContrast: true }`.

For live meet data, swim results, and incoming watch times, bypass React Context entirely. Let `meetCache` hold the raw data, and let **Remix loaders + useRevalidator()** drive the UI updates.

User prompt: Next question: what's the best way to represent the order of events and heats within the meet? Currently my meet object has an array of event definitions (ie stroke, gender, distance, type), entries, teams, swims (which are what are seeded and include the heat/lane information as discussed above)... should there be a races array or something?

Response: You do **not** need a separate races entity. Modeling a dedicated array or table for "races" creates redundant state that must be kept in sync with both your events and your swims.

The standard, battle-tested domain model in swimming establishes order through a simple hierarchy: **Event Sequence Number** → **Heat Number** → **Lane Number**.

The Natural Timeline: Events Order Heats, Heats Order Swims

In swimming, the physical progression on deck is strictly linear:

1. Event 1 (e.g., Girls 200 Medley Relay), Heat 1
2. Event 1, Heat 2
3. Event 2 (e.g., Boys 200 Medley Relay), Heat 1
4. ...and so on.

You already have the pieces to represent this without adding a middleman table:

```
Event (has `order_index` or `event_number`)
└─ Swims (have `heat_number` and `lane_number`)
```

A "heat" is simply a group of Swim rows that share the same (event_id, heat_number).

How to Model the Order

1. In the Event Model: Use an Explicit order_index

Never rely on array position in JSON or primary key order in a database. High school dual meets follow a standard 12-event order, but JV events, exhibitions, or duals with diving breaks can alter the sequence.

```
interface MeetEvent {
  id: string;
  meetId: string;
  eventNumber: number;      // e.g., 1, 2, 3 (display label: "Event 3")
  orderIndex: number;       // Explicit sequence: 10, 20, 30... allows reordering/insertions
  gender: "boys" | "girls" | "mixed";
  stroke: "free" | "back" | "breast" | "fly" | "im" | "medley";
  distance: number;         // e.g., 50, 100, 200, 500
  isRelay: boolean;
  status: "upcoming" | "in_progress" | "completed";
}
```

Tip: Giving orderIndex steps of 10 (or floating points) lets you insert a "Diving Break" or an unexpected JV relay heat without having to re-index every subsequent event in the database.

2. In the Swim Model: Group by heat_number

Because your Swim records already hold event_id, heat_number, and lane_number, they inherently define the physical schedule:

```
interface Swim {
  id: string;
  eventId: string;
  heatNumber: number;       // 1, 2, 3...
  laneNumber: number;       // 1, 2, 3, 4, 5, 6
  athleteId?: string;
  status: "pending" | "official" | "dq" | "dns";
  // ...watches, officialTimeMs, etc.
}
```

How to Derive the Meet Schedule (No Extra Table)

Whenever your UI or Admin view needs the complete chronological schedule of the meet (e.g., to power "Current Heat", "Next Heat", or the volunteer's heat stepper), derive it with a single query or a memoized selector:

In SQLite (inside your Durable Object):

```
SELECT
  e.id AS event_id,
  e.event_number,
  e.order_index,
  s.heat_number,
  COUNT(s.id) AS total_lanes,
  COUNT(CASE WHEN s.status = 'official' THEN 1 END) AS completed_lanes
FROM events e
JOIN swims s ON s.event_id = e.id
```

```
GROUP BY e.id, s.heat_number
ORDER BY e.order_index ASC, s.heat_number ASC;
```

On the Client (derived from your meet manifest):

```
interface ScheduledHeat {
  eventId: string;
  eventNumber: number;
  eventName: string;
  heatNumber: number;
  totalHeatsInEvent: number;
  isComplete: boolean;
}

export function getChronologicalHeats(events: MeetEvent[], swims: Swim[]): ScheduledHeat[] {
  // Sort events by designated sequence
  const sortedEvents = [...events].sort((a, b) => a.orderIndex - b.orderIndex);

  return sortedEvents.flatMap((event) => {
    const eventSwims = swims.filter((s) => s.eventId === event.id);
    const heatNumbers = Array.from(new Set(eventSwims.map((s) => s.heatNumber))).sort((a, b) => a - b);

    return heatNumbers.map((heatNum) => {
      const heatSwims = eventSwims.filter((s) => s.heatNumber === heatNum);
      const isComplete = heatSwims.length > 0 && heatSwims.every((s) => s.status === "official");

      return {
        eventId: event.id,
        eventNumber: event.eventNumber,
        eventName: `${event.gender} ${event.distance} ${event.stroke}`,
        heatNumber: heatNum,
        totalHeatsInEvent: heatNumbers.length,
        isComplete,
      };
    });
  });
}
```

What State Actually Belongs on the Meet Object?

The meet level doesn't need a list of races; it only needs a pointer to the **current active deck position**:

```
interface Meet {
  id: string;
  name: string;
  isLive: boolean;
  // Current deck pointer:
  currentEventId?: string;
  currentHeatNumber?: number;
}
```

Why This Is the Cleanest Approach

1. **Zero Sync Overhead:** If an extra JV heat is added to Event 4, you simply insert rows into swims with `heat_number: 3`. You don't have to remember to update a separate races registry.
2. **Deterministic Stepping:** "Advance to Next Heat" is just finding the next index in your sorted list of (`orderIndex`, `heatNumber`).
3. **Natural Filtering:**
 - Want everything for a lane timer? `swims.filter(s => s.laneNumber === myLane)`.
 - Want the live heat for the starter? `swims.filter(s => s.eventId === currentEventId && s.heatNumber === currentHeatNumber)`.

User prompt: I like the idea of current heat and current event number... But I've been concerned about maintaining state like this and ensuring things work correctly with intermittent Internet... if the current event and heat are broadcast VMI socket, but someone is off-line I guess the timer would just have to manually advance and maintain that state themselves?

Response: That intuition is spot on. If you treat `currentEventId` and `currentHeatNumber` as an **authoritative lock** that drives the client UI, intermittent connectivity will create pure chaos on deck:

- A volunteer's phone loses signal behind the diving well.

- The admin advances to Heat 3.
- The timer's phone never gets the packet and remains stuck on Heat 2—or worse, snaps forward unexpectedly right as they are about to tap the screen.

The solution is to decouple **Timer Autonomous Mode** from **Admin Deck Pointer**.

The Architecture: "Advisory" Server Pointers + Local Steppers

The timer's UI must **never** be forcefully pushed forward or blocked by an incoming network packet.

Instead:

1. The Timer's State is Local and Autonomous:

The primary driver for a lane timer is their own sequential workflow:

Start → Stop → Submit & Advance → (Heat $N + 1$)

Advancing from one heat to the next is a purely local client-side navigation (`navigate(...)`). It never waits on a network round-trip, an admin signal, or a WebSocket ACK.

2. The Server's Broadcast is Strictly "Advisory":

The server broadcasts `ACTIVE_DECK_POSITION: { eventId, heatNumber }`.

- If a timer is in sync, their local heat matches the broadcast, and nothing happens.
- If a timer drops offline for three heats, comes back online, and is sitting on Heat 2 while the pool is on Heat 5, you **do not** automatically jump their screen (which could destroy an active stopwatch).
- You simply display an unobtrusive, dismissible banner at the top of their screen:

"Pool is currently on Event 4, Heat 5. [Jump to Live Heat]"

How to Model the Route & State to Make This Bulletproof

Because your URL routing already models physical position: `/meets/:id/timer/events/:event/heats/:heat/lanes/:lane`

The URL *is* the timer's current state. They don't need a reactive variable in memory.

1. The Autonomous "Submit & Advance" Action

When the timer hits submit, they advance along their local timeline derived from the meet manifest already in `meetCache`:

```
// Inside your lane timer component:
function handleFormSubmitAndAdvance() {
  // 1. Fire off the submission (HTTP POST + cookie queue)
  submitForm();

  // 2. Compute the next local chronological slot from cached manifest
  const next = getNextScheduledHeat(meetManifest, currentEventId, currentHeatNumber);

  if (next) {
    // 3. Navigate immediately — 0ms latency, zero network dependency
    navigate(`/meets/${meetId}/timer/events/${next.eventId}/heats/${next.heatNumber}/lanes/${myLane}`);
  } else {
    navigate(`/meets/${meetId}/timer/finished`);
  }
}
```

2. The Advisory Sync Banner

A small listener purely compares the URL params against the latest advisory socket packet:

```
function AdvisoryDeckSync({ currentEventId, currentHeatNumber, onJump }: Props) {
  // Latest position received from DO broadcast (or null if offline)
  const deckPosition = useLatestDeckPosition();

  if (!deckPosition) return null; // Offline? No problem, volunteer stays in autonomous mode.

  const isDiverged =
    deckPosition.eventId !== currentEventId ||
    deckPosition.heatNumber !== currentHeatNumber;

  if (!isDiverged) return null;
```

```

return (
  <div className="bg-amber-500/90 text-slate-950 px-3 py-1.5 text-xs font-semibold flex justify-between item">
    <span>Pool is on {deckPosition.eventName} • Heat {deckPosition.heatNumber}</span>
    <button
      onClick={() => onJump(deckPosition.eventId, deckPosition.heatNumber)}
      className="ml-3 bg-slate-900 text-white px-2 py-0.5 rounded text-[11px] uppercase tracking-wider"
    >
      Jump →
    </button>
  </div>
);
}

```

Why This Inversion Eliminates Deck Headaches

- **No Locked Screens:** If the gym Wi-Fi dies completely, timers can run the entire 24-heat meet from start to finish without touching a reconnect button. All submissions queue in cookies, and their heat stepper just increments locally.
- **No Accidental Overwrites:** If a volunteer is reviewing a touch time from Heat 2 and the starter blows the horn for Heat 3, an authoritative broadcast won't pull the rug out from under their fingers.
- **Easy Recovery from Human Error:** If a volunteer accidentally taps "Submit & Advance" twice and finds themselves on Heat 4 while the swimmers on the blocks are Heat 3, they have a clear "Previous Heat / Jump to Live" control to step back manually.
- **Great for Alternating Starts:** If the starter runs "flyover starts" or alternates boys and girls heats, the local scheduler advances in whatever sequence the meet was programmed, but the advisory banner keeps volunteers oriented if the deck referee decides to skip or reorder a heat.

User prompt: does this seem like a clean data model for the client, then? (note definitions aren't any valid syntax, combo of sql and typescript) and, we'd have helper functions like above to get current event/next event? watch { id TEXT PRIMARY KEY, swim_id TEXT NOT NULL, submitted_by TEXT NOT NULL, user_id TEXT, role TEXT NOT NULL DEFAULT 'timer', slot INTEGER NOT NULL DEFAULT 1, time_ms INTEGER, started_at INTEGER, stopped_at INTEGER, submitted_at INTEGER NOT NULL } swim { id TEXT PRIMARY KEY, event_id TEXT NOT NULL, heat INTEGER NOT NULL, lane INTEGER NOT NULL, athlete_id TEXT NOT NULL, athlete_name TEXT NOT NULL DEFAULT "", athlete_team TEXT NOT NULL DEFAULT "", exhibition INTEGER NOT NULL DEFAULT 0, status TEXT, official_time_ms INTEGER, decided_at INTEGER, decided_by TEXT } heat { lanes : swim[], } event { id TEXT PRIMARY KEY, position INTEGER NOT NULL, distance INTEGER NOT NULL, stroke TEXT NOT NULL, gender TEXT NOT NULL, name TEXT heats : heat[] } meet { details events : event[] } const currentEvent = meet.events[101] const currentHeat = meet.events[101].heats[1] const currentSwim = meet.events[101].heats[1].lanes[1]

Response: The nested tree structure (meet.events[] . heats[] . lanes[]) is intuitive to look at, but using array indexing like meet.events[101].heats[1].lanes[1] introduces subtle bugs in JavaScript:

1. Array Index vs. Business Number Mismatch:

In JS arrays, meet.events[101] accesses the **102nd element**, not Event #101.

Similarly, lanes[1] accesses the *second* lane (0-indexed), not Lane 1. If an event has 3 heats, heats[1] is actually Heat 2. This off-by-one impedance mismatch creates bugs across forms, route params, and display labels.

2. Sparse Array Allocation:

If you have Event 1, Event 2, and Event 101, an array lookup either requires sparse array allocation (new Array(102)) or sorting/finding by .find(e => e.position === 101).

3. Immutability & WebSocket Patch Friction:

When a WebSocket packet arrives for Heat 2, Lane 4, updating deeply nested arrays (events -> heats -> lanes -> swim) requires either multi-level object spreads or a heavy immutability library.

The Clean Client Representation: Normalized Maps or Records

In a reactive client cache, flat records or lookup dictionaries are faster, cleaner to update on incoming WebSocket packets, and eliminate all off-by-one index arithmetic.

1. The Normalized Entity Shape

```

export interface Watch {
  id: string;
  swimId: string;
  submittedBy: string;
  userId?: string;
  role: "timer" | "chief_timer" | "admin";
  slot: number; // Watch 1, 2, or 3
  timeMs?: number;
  startedAt?: number;
  stoppedAt?: number;
  submittedAt: number;
}

```

```

}

export interface Swim {
  id: string;
  eventId: string;
  heat: number;           // 1-based (Heat 1, 2, 3)
  lane: number;           // 1-based (Lane 1, 2, 3, 4, 5, 6)
  athleteId?: string;     // Optional for open lanes
  athleteName?: string;
  athleteTeam?: string;
  exhibition: boolean;
  status: "pending" | "official" | "dq" | "dns";
  officialTimeMs?: number;
  decidedAt?: number;
  decidedBy?: string;
  watches?: Watch[];      // Hydrated watches for easy UI display
}

export interface MeetEvent {
  id: string;
  position: number;       // Chronological order: 1, 2, 3 (or 10, 20, 30)
  eventNumber: number;    // e.g. 101
  distance: number;       // 50, 100, 200
  stroke: "free" | "back" | "breast" | "fly" | "im" | "medley";
  gender: "boys" | "girls" | "mixed";
  name?: string;
}

export interface MeetManifest {
  id: string;
  name: string;
  isLive: boolean;
  events: Record<string, MeetEvent>; // Keyed by eventId
  swims: Record<string, Swim>;       // Keyed by `${eventId}_h${heat}_l${lane}`
}

```

2. Direct O(1) Lookups & Clean Helper Functions

By composite-keying the swims dictionary as `\${eventId}\_h\${heat}\_l\${lane}`, lookups in your route components are direct:

```

// Look up the exact physical swim for a lane timer in 0ms:
const swimKey = `${eventId}_h${heatNumber}_l${laneNumber}`;
const currentSwim = meet.swims[swimKey];

```

Helper Selectors

```

// 1. Get ordered list of events for the meet
export function getSortedEvents(meet: MeetManifest): MeetEvent[] {
  return Object.values(meet.events).sort((a, b) => a.position - b.position);
}

// 2. Get all swims for a specific heat (ordered by lane)
export function getHeatSwims(meet: MeetManifest, eventId: string, heatNumber: number): Swim[] {
  return Object.values(meet.swims)
    .filter((s) => s.eventId === eventId && s.heat === heatNumber)
    .sort((a, b) => a.lane - b.lane);
}

// 3. Find the total number of heats in an event
export function getTotalHeatsForEvent(meet: MeetManifest, eventId: string): number {
  const heats = new Set(
    Object.values(meet.swims)
      .filter((s) => s.eventId === eventId)
      .map((s) => s.heat)
  );
  return heats.size;
}

// 4. Stepper: Calculate the next sequential heat/event
export function getNextHeat(
  meet: MeetManifest,
  currentEventId: string,
  currentHeat: number
): { eventId: string; heat: number } | null {

```

```

const totalHeats = getTotalHeatsForEvent(meet, currentEventId);

// Still more heats in the current event?
if (currentHeat < totalHeats) {
  return { eventId: currentEventId, heat: currentHeat + 1 };
}

// Move to the next event in the schedule
const sortedEvents = getSortedEvents(meet);
const currentEventIdx = sortedEvents.findIndex((e) => e.id === currentEventId);

if (currentEventIdx !== -1 && currentEventIdx + 1 < sortedEvents.length) {
  const nextEvent = sortedEvents[currentEventIdx + 1];
  return { eventId: nextEvent.id, heat: 1 };
}

// Reached end of meet
return null;
}

```

Why This Simplifies the Client

- **Zero Array Index Math:** You never have to do `lanes[lane - 1]` or wonder if `heats[1]` means Heat 1 or Heat 2.
- **Instant WebSocket Patches:** When a timer submits a watch time, your socket listener updates the local cache in a single line without traversing nested objects:


```
meet.swims[`${eventId}_h${heat}_l${lane}`].watches.push(incomingWatch);
```
- **Robust Component Scoping:** The leaf timer route (`/meets/:id/timer/events/:event/heats/:heat/lanes/:lane`) grabs its exact swim slice in one call:


```
const currentSwim = meet.swims[`${params.event}_h${params.heat}_l${params.lane}`];
```

User prompt: would it make sense to have a dedicated `SwimKey` type like `{ heat: number, event: number, lane: number}` and have `swims` be a `Record<SwimKey, Swim>`?

Response: In JavaScript and TypeScript, **objects cannot be used as keys in a plain `Record<K, V>` or plain object `{}`**.

If you define:

```

type SwimKey = { eventId: string; heat: number; lane: number };

// TypeScript error: Type 'SwimKey' is not assignable to type 'string | number | symbol'
type SwimRecord = Record<SwimKey, Swim>;

```

JavaScript coerces any non-primitive key on an object to a string:

```

const obj = {};
obj[{ eventId: "ev1", heat: 1, lane: 1 }] = swim1;
obj[{ eventId: "ev2", heat: 2, lane: 2 }] = swim2;

// obj now only has ONE key: "[object Object]"
// The second assignment overwrote the first!

```

Your 3 Clean Options

1. Template Literal String Key (Recommended)

You can still get **full TypeScript autocomplete and type-safety** by defining a branded template literal string type. This works natively with plain objects, JSON serialization, and `localStorage`.

```

export type SwimKey = `${string}:h${number}:l${number}`;

export function makeSwimKey(eventId: string, heat: number, lane: number): SwimKey {
  return `${eventId}:h${heat}:l${lane}`;
}

export function parseSwimKey(key: SwimKey): { eventId: string; heat: number; lane: number } {
  const [eventId, h, l] = key.split(":");
  return {
    eventId,
    heat: Number(h.replace("h", "")),
    lane: Number(l.replace("l", "")),
  };
}

```

```
};
}

// Full type safety on standard Record:
export type SwimMap = Record<SwimKey, Swim>;
```

Usage:

```
// Lookup in route or component
const key = makeSwimKey(params.event!, Number(params.heat), Number(params.lane));
const swim = meet.swims[key];
```

- **Why this is great:** It serializes directly to/from JSON (`JSON.stringify(meet)`), stores in `localStorage`, and requires zero transformation when sent over WebSockets or stored in SQLite.

2. JavaScript Map with String Keys

If you prefer a Map over a plain object, pair it with the string serializer:

```
export class SwimStore {
  private items = new Map<SwimKey, Swim>();

  get(eventId: string, heat: number, lane: number): Swim | undefined {
    return this.items.get(makeSwimKey(eventId, heat, lane));
  }

  set(swim: Swim): void {
    this.items.set(makeSwimKey(swim.eventId, swim.heat, swim.lane), swim);
  }
}
```

3. Why Map<SwimKey, Swim> with Object References Fails

You *can* pass objects into a native ES6 Map<SwimKey, Swim>, but lookup is by **object identity (reference equality)**, not structural equality:

```
const map = new Map<SwimKey, Swim>();

const keyA = { eventId: "ev101", heat: 1, lane: 4 };
map.set(keyA, swim);

// In a component later:
const lookupKey = { eventId: "ev101", heat: 1, lane: 4 };
map.get(lookupKey); // -> undefined! (Different object references in memory)
```

To make that work, you would have to maintain a shared reference cache or interning pool, which adds needless overhead.

The Verdict

Use **Option 1**: a strongly-typed template literal `SwimKey (${string}:h${number}:l${number})` with a plain `Record<SwimKey, Swim>`. It gives you compile-time safety, fast $O(1)$ lookups, and serialization with zero runtime overhead.

User prompt: in the above, for something like `getTotalHeatsForEvent` wouldn't i want to memoize that?

Response: Yes, you definitely want to avoid recalculating that on every render—but **how** you memoize it depends on where that data lives.

Because `meet.swims` contains every lane in every heat across the entire meet, calling:

```
Object.values(meet.swims).filter(...)
```

iterates through hundreds of objects and allocates new arrays and sets every single time a component renders or revalidates.

The Best Approach: Pre-Calculate at Ingestion (Zero-Cost Lookups)

Rather than memoizing in React hooks (`useMemo`), the cleanest approach in a normalized data model is to **compute and store structural metadata once** when the meet manifest is parsed or received.

Total heats for an event is a structural property that rarely changes once seeded. Put it directly on the `MeetEvent` object:

```
export interface MeetEvent {
  id: string;
  position: number;
  eventNumber: number;
  name: string;
```

```
totalHeats: number; // <-- Compute once upon receiving the manifest
}
```

When your loader or WebSocket receives the meet payload and hydrates the cache:

```
// Inside your manifest normalization step:
export function hydrateManifest(raw: RawMeetPayload): MeetManifest {
  const swims: Record<SwimKey, Swim> = {};
  const heatsPerEvent: Record<string, Set<number>> = {};

  // 1. Ingest swims and collect distinct heats in one pass
  for (const swim of raw.swims) {
    const key = toSwimKey({ eventId: swim.eventId, heat: swim.heat, lane: swim.lane });
    swims[key] = swim;

    if (!heatsPerEvent[swim.eventId]) heatsPerEvent[swim.eventId] = new Set();
    heatsPerEvent[swim.eventId].add(swim.heat);
  }

  // 2. Decorate events with their pre-calculated totalHeats
  const events: Record<string, MeetEvent> = {};
  for (const ev of raw.events) {
    events[ev.id] = {
      ...ev,
      totalHeats: heatsPerEvent[ev.id]?.size ?? 1,
    };
  }

  return { id: raw.id, name: raw.name, isLive: raw.isLive, events, swims };
}
```

Now getTotalHeatsForEvent becomes an instant $O(1)$ property access:

```
const totalHeats = meet.events[eventId]?.totalHeats ?? 1;
```

Zero hook overhead, zero cache invalidation bugs, and fully available even outside React components (e.g., inside client actions, background queues, or route loaders).

If You Keep It Derived: Two Clean Memoization Patterns

If you prefer to keep the raw data model unaugmented and derive values on the fly:

1. Inside React Components: useMemo

If called inside a component (like a header showing "Heat 2 of 4"):

```
const totalHeats = useMemo(() => {
  return getTotalHeatsForEvent(meet, eventId);
}, [meet.swims, eventId]);
```

Note: Be careful with dependencies. If `meet.swims` updates reference on every incoming watch touch, this will still recalculate often unless scoped properly.

2. External Selector Cache (e.g., WeakMap / LRU)

If you need helper functions like `getNextHeat()` inside non-component code (like `clientAction` or utility helpers):

```
// Cache keyed by the swims reference
const heatsCache = new WeakMap<Record<SwimKey, Swim>, Map<string, number>>();

export function getTotalHeatsForEvent(meet: MeetManifest, eventId: string): number {
  let eventMap = heatsCache.get(meet.swims);
  if (!eventMap) {
    eventMap = new Map<string, number>();
    heatsCache.set(meet.swims, eventMap);
  }

  if (eventMap.has(eventId)) {
    return eventMap.get(eventId)!;
  }

  const heats = new Set<number>();
  for (const swim of Object.values(meet.swims)) {
    if (swim.eventId === eventId) {
      heats.add(swim.heat);
    }
  }

  return heats.size;
}
```

```

    }
  }

  const count = heats.size;
  eventMap.set(eventId, count);
  return count;
}

```

Verdict

Stashing `totalHeats` directly on `MeetEvent` when building or caching the manifest is the most idiomatic. It eliminates array scanning entirely and keeps helper functions fast and predictable across all views.

User prompt: related data model question - should all my client POCOs for event, heat, etc have a `meet_id` property? it's always going to be the same and is always derivable from the URL... that's definitely need in each row in sql tables, but is it worth having on the POCOs?

Response: **No, strip `meetId` from the client POCOs.**

In your SQLite tables on Cloudflare DO or D1, `meet_id` is essential for indexing, foreign keys, partition isolation, and data integrity. On the client, however, stamping `meetId` onto every `Event`, `Swim`, and `Watch` is redundant overhead that works against your offline-first architecture.

Why Dropping `meetId` from Client POCOs is the Right Call

1. Payload and Storage Footprint:

In a 48-event high school dual meet with 3 heats per event and 6 lanes per heat, you have over 800 `Swim` and `Watch` records. Serializing a 36-character UUID ("`meetId`": "`a3f89b12-...`") across every single entity inflates your WebSocket message payloads, initial SSR transfer size, and `localStorage` JSON cache by tens of kilobytes of pure duplication.

2. The URL and Route Tree Are the Root Context:

Your routes are already structured as:

```
/meets/:id/...
```

Every component, loader, and `clientAction` already knows its `meetId` via `params.id`. It is impossible for an event or swim rendered on `/meets/123/timer/...` to belong to meet 456.

3. Natural Containment in `MeetManifest`:

Your client data model already groups everything under a single root object:

```

export interface MeetManifest {
  id: string; // <--- Here is the only place meetId needs to live
  name: string;
  isLive: boolean;
  events: Record<string, MeetEvent>;
  swims: Record<SwimKey, Swim>;
}

```

Having `meet.swims[key].meetId === meet.id` is redundant property nesting.

The Only Exception: Outgoing Queue Payloads

The only time your client needs to care about `meetId` on an entity level is when **serializing an outbound mutation** to the server:

```

// Submitting from clientAction to the D0 / Server action:
const payload = {
  meetId: params.id, // Injected from route context at submission time
  eventId: swim.eventId,
  heat: swim.heat,
  lane: swim.lane,
  watches: [...],
};

```

Your form submission handler or HTTP POST client injects `params.id` into the payload at the boundary. The client models themselves stay lean and clean:

```

export interface MeetEvent {
  id: string;
  position: number;
  eventNumber: number;
  distance: number;
  stroke: "free" | "back" | "breast" | "fly" | "im" | "medley";
}

```

```

    gender: "boys" | "girls" | "mixed";
    totalHeats: number;
  }

export interface Swim {
  id: string;
  eventId: string;
  heat: number;
  lane: number;
  athleteId?: string;
  athleteName?: string;
  athleteTeam?: string;
  exhibition: boolean;
  status: "pending" | "official" | "dq" | "dns";
  officialTimeMs?: number;
  watches?: Watch[];
}

```

Keep relational database foreign keys in SQL; keep client POCOs scoped cleanly to the document hierarchy they inhabit.

User prompt: coming back to my views - eg i have "timer-shell.tsx", "timer-clipboard.tsx", and "timer-stopwatch.tsx"... clipboard and stopboard are the two different views/modes a timer may work in. the shell is the shared layout file, which has a loader that loads all data for the current meet... idea is the child routes then rely on that data for instant navigation, even offline, and that the parent/shell route is where our websocket component (defined above) exists, to receive messages broadcast from our durable object and update our local cache. now - how do i ensure my child views update as messages come in? should i use generic context? remix's Outlet context feature? remix useRevalidate called by the shell upon receiving new data?

Response: The cleanest, most idiomatic pattern here is **meetCache + useRevalidator()**, paired with **useRouteLoaderData()** in your child views.

You should **avoid generic React Context** (for the re-render performance reasons we broke down earlier) and **avoid Remix's useOutletContext()** for live data.

Why useOutletContext() Breaks Down Here

Remix's `<Outlet context="{...}" />` is handy for passing simple, stable callbacks down to children, but using it for a stream of live data creates two friction points:

1. Top-Level Re-Render Cascade:

Passing live data via `<Outlet context="{ { 'meet' } }" />` requires the shell to hold `meet` in local React state (`useState`). Every incoming WebSocket message forces `setMeet(...)` at the very root of your layout, re-rendering the entire shell, its headers, its navigation bars, and whatever child view is mounted.

2. Coupling and Boilerplate:

If you ever embed the clipboard or stopwatch components inside a modal, split-view, or tester route that isn't a direct child of that exact `<Outlet>`, `useOutletContext()` throws or returns undefined.

The Clean Pattern: meetCache + useRevalidator()

Let Remix's loader pipeline do what it was designed to do.

WebSocket Packet Arrives in `<LiveMeetSync />` (inside shell)

1. Update in-memory singleton: ``meetCache.applyPatch(msg)``
2. Call ``revalidator.revalidate()``
3. Shell's ``clientLoader`` re-runs instantly (0ms read from ``meetCache``)
4. Child views (``timer-clipboard``, ``timer-stopwatch``) receive the fresh data via: ``useRouteLoaderData("routes/meets.$id.timer")``

Why This Is Superior:

- **Zero Hook Complexity:** The child views don't need socket listeners, custom hooks, or context subscriptions. They simply treat `useRouteLoaderData(...)` as their source of truth.
- **Completely Decoupled:** The child views don't care *how* the data was updated—whether from a cold SSR paint, an offline local write, or an incoming live WebSocket patch.

- **No UI Jitter on High-Frequency Updates:** Because `clientLoader` returns synchronously from the in-memory cache, revalidation completes in the same tick without loading spinners or navigation flashes.

Concrete Implementation

1. The Parent Shell (`app/routes/meets.$id.timer.tsx`)

The shell owns the data loader, caches it locally, renders the live sync provider, and mounts the child routes via `<Outlet/>`.

```
// app/routes/meets.$id.timer.tsx
import type { LoaderFunctionArgs, ClientLoaderFunctionArgs } from "@remix-run/cloudflare";
import { Outlet, useLoaderData, useRevalidator } from "@remix-run/react";
import { useEffect } from "react";
import { meetCache, type MeetManifest } from "~/lib/meetCache";

export async function loader({ params, context }: LoaderFunctionArgs) {
  const meetId = params.id!;
  const doId = context.env.MEET_DO.idFromName(meetId);
  const stub = context.env.MEET_DO.get(doId);
  const meet: MeetManifest = await stub.getMeetManifest();
  return { meet };
}

export async function clientLoader({ params, serverLoader }: ClientLoaderFunctionArgs) {
  const meetId = params.id!;
  const cached = meetCache.getManifest(meetId);

  // Stale-While-Revalidate if cached in memory/localStorage
  if (cached) {
    serverLoader()
      .then((fresh) => {
        if (fresh?.meet) meetCache.saveManifest(meetId, fresh.meet);
      })
      .catch(() => {});
    return { meet: cached };
  }

  const fresh = await serverLoader();
  meetCache.saveManifest(meetId, fresh.meet);
  return fresh;
}

export default function TimerShell() {
  const { meet } = useLoaderData<typeof loader>();

  return (
    <div className="min-h-screen bg-slate-950 text-white flex flex-col">
      {/* Headless socket listener living safely at the layout boundary */}
      {meet.isLive && <LiveMeetSync meetId={meet.id} />}

      {/* Child views render here */}
      <main className="flex-1 flex flex-col">
        <Outlet />
      </main>
    </div>
  );
}

function LiveMeetSync({ meetId }: { meetId: string }) {
  const revalidator = useRevalidator();

  useEffect(() => {
    const protocol = window.location.protocol === "https:" ? "wss:" : "ws:";
    const ws = new WebSocket(`${protocol}/${window.location.host}/api/meets/${meetId}/live`);

    ws.onmessage = (event) => {
      const msg = JSON.parse(event.data);

      // Fast patch directly into the singleton memory cache
      if (meetCache.applyPatch(meetId, msg)) {
        // Nudge Remix to refresh all active useRouteLoaderData listeners
        revalidator.revalidate();
      }
    };
  });
}
```

```

    return () => ws.close();
  }, [meetId, revalidator]);

  return null;
}

```

2. The Child Mode: Stopwatch View (timer-stopwatch.tsx)

Notice how clean this is—it pulls straight from the shell's loader ID and derives what it needs in one line:

```

// app/routes/meets.$id.timer.events.$event.heats.$heat.lanes.$lane.stopwatch.tsx
import { useParams, useRouteLoaderData } from "@remix-run/react";
import type { loader as shellLoader } from "~/routes/meets.$id.timer";
import { toSwimKey } from "~/lib/meetCache";

export default function TimerStopwatchView() {
  const params = useParams();

  // Read directly from the parent shell loader
  const rootData = useRouteLoaderData<typeof shellLoader>("routes/meets.$id.timer");
  const meet = rootData?.meet;

  const currentEventId = params.event!;
  const currentHeat = Number(params.heat);
  const currentLane = Number(params.lane);

  // 0(1) direct slot lookup
  const swimKey = toSwimKey({ eventId: currentEventId, heat: currentHeat, lane: currentLane });
  const currentSwim = meet?.swims[swimKey];
  const currentEvent = meet?.events[currentEventId];

  return (
    <div className="flex-1 flex flex-col p-4 select-none">
      <div className="flex justify-between items-baseline border-b border-slate-800 pb-2">
        <h2 className="text-sm font-semibold uppercase text-teal-400">
          {currentEvent?.name}
        </h2>
        <span className="text-xs text-slate-400">
          Heat {currentHeat} • Lane {currentLane}
        </span>
      </div>

      <div className="py-2">
        <span className="text-lg font-bold text-white">
          {currentSwim?.athleteName || "Open Lane"}
        </span>
        {currentSwim?.exhibition && (
          <span className="ml-2 text-xs bg-amber-500/20 text-amber-300 px-1.5 py-0.5 rounded">
            Exhibition
          </span>
        )}
      </div>

      {/* Stopwatch triggers, start/stop touches, and form submission */}
    </div>
  );
}

```

3. The Child Mode: Clipboard View (timer-clipboard.tsx)

The clipboard view might want to see the whole heat or previous heats for that lane to review times:

```

// app/routes/meets.$id.timer.lanes.$lane.clipboard.tsx
import { useParams, useRouteLoaderData } from "@remix-run/react";
import type { loader as shellLoader } from "~/routes/meets.$id.timer";

export default function TimerClipboardView() {
  const { lane } = useParams();
  const rootData = useRouteLoaderData<typeof shellLoader>("routes/meets.$id.timer");
  const meet = rootData?.meet;

  const currentLane = Number(lane);

```

```
// Filter all swims assigned to this lane across the meet
const laneSwims = Object.values(meet?.swims ?? {}).
  .filter((s) => s.lane === currentLane);

return (
  <div className="p-4 space-y-3">
    <h1 className="text-xl font-bold">Lane {currentLane} Paper Sheet Backup</h1>
    <div className="divide-y divide-slate-800">
      {laneSwims.map((swim) => (
        <div key={swim.id} className="py-2 flex justify-between items-center text-sm">
          <span>Heat {swim.heat}: {swim.athleteName} || "Unassigned"</span>
          <span className="font-mono">
            {swim.officialTimeMs ? (swim.officialTimeMs / 1000).toFixed(2) : "-"}
          </span>
        </div>
      ))}
    </div>
  </div>
);
}
```

What Makes This Setup Ideal

- 1. Zero Route Re-Fetching on Offline:** When `revalidator.revalidate()` fires, `clientLoader` runs on the client. Because `meetCache.getManifest()` returns from memory, it never touches the network unless explicitly needed.
- 2. Instant Transitions Between Modes:** Flipping between "Stopwatch Mode" and "Clipboard Mode" is an instant client-side route transition with 0ms load time—both modes read from the exact same in-memory `useRouteLoaderData` reference.
- 3. No State Sync Bugs:** There are no `useEffect` synchronization loops between props and state, and no risks of a React Context provider re-rendering components mid-touch.

User prompt: beautiful... using this approach, should i cache the entire meet object (ie the key is basically the meetId and the response is our whole meet with events, athletes, swims, etc)? or, cache things separately? worth going down to individual objects, like each swim, each watch, etc?

Response: **Cache the entire meet document as a single composite object under `meetId`.** Do not break the cache down into granular keys for individual swims, watches, or events.

Treating the meet as a single cohesive aggregate matches the physical scale of swimming and avoids the classic traps of granular client-side key-value stores.

Why Granular Caching (swim:123, watch:456) Breaks Down

Splitting your client cache into hundreds of tiny discrete keys sounds modern, but on a pool deck it introduces significant overhead:

1. The Reassembly Tax:

If every swim is its own key, every page transition, revalidation, or clipboard view requires pulling 50 to 800 individual keys out of storage/memory, re-filtering them, and stitching them into an event tree.

2. Atomic Consistency vs. Tearing:

If a reseed occurs, or an admin moves Heat 2 to Heat 3, updating 24 separate `swim:*` keys individually risks "cache tearing"—where a component reads half of the newly moved swims and half of the old ones during a render cycle.

3. Garbage Collection & Orphaned Keys:

When a heat is scratched or an event is reseeded, granular caches leave dead rows behind in `localStorage` unless you write a custom relational cleanup engine on the client.

4. localStorage I/O Bottlenecks:

Writing 50 distinct keys to browser storage during initial sync triggers 50 synchronous disk write barriers. Writing a single serialized meet object triggers **one**.

The Reality of Meet Scale: Why "One Big Object" Wins

In web development, we are conditioned to fear large JSON blobs. But let's look at the actual math for a standard dual meet:

- 12 to 24 Events
- ~3 Heats per event
- 6 Lanes per heat

- $\approx$ 200 to 400 total **Swim** records
- $\approx$ 800 total **Watch** records

Serialized as JSON, an entire high school dual meet with all events, athletes, seeded swims, and recorded watches is **between 150 KB and 350 KB**.

That is smaller than a single hero banner image.

- Parsing 250 KB of JSON in JavaScript takes **1 to 3 milliseconds**.
- Storing 250 KB fits easily inside the browser's 5 MB `localStorage` quota.
- Pulling it from an in-memory Map is **instantaneous (0ms)**.

The Structure: One In-Memory Aggregate with Granular Socket Patching

Keep the storage and loader representation coarse, but apply incoming socket patches **mutatively in memory**:

```
// app/lib/meetCache.ts

class MeetCacheManager {
  // In-memory master manifest: keyed purely by meetId
  private manifests = new Map<string, MeetManifest>();

  getManifest(meetId: string): MeetManifest | null {
    if (this.manifests.has(meetId)) {
      return this.manifests.get(meetId)!;
    }
    if (typeof window !== "undefined") {
      const raw = localStorage.getItem(`meet:${meetId}`);
      if (raw) {
        try {
          const parsed = JSON.parse(raw);
          this.manifests.set(meetId, parsed);
          return parsed;
        } catch (_) {}
      }
    }
    return null;
  }

  saveManifest(meetId: string, manifest: MeetManifest) {
    this.manifests.set(meetId, manifest);
    if (typeof window !== "undefined") {
      try {
        localStorage.setItem(`meet:${meetId}`, JSON.stringify(manifest));
      } catch (_) {}
    }
  }

  /**
   * Fast mutative patch: updates the target swim inside the existing meet aggregate.
   * Zero serialization cost; prepares the object for useRevalidator().
   */
  applyPatch(meetId: string, msg: any): boolean {
    const meet = this.getManifest(meetId);
    if (!meet) return false;

    switch (msg.type) {
      case "SWIM_TOUCH_RECORDED": {
        const key = `${msg.eventId}:h${msg.heat}:l${msg.lane}` as SwimKey;
        const swim = meet.swims[key];
        if (swim) {
          if (!swim.watches) swim.watches = [];
          swim.watches.push(msg.watch);
          swim.officialTimeMs = msg.officialTimeMs;
          swim.status = msg.status;

          // Debounce or lazily flush to localStorage so disk writes don't lag touch listeners
          this.debounceSave(meetId, meet);
          return true;
        }
      }
    }
    break;
  }
}
```

```

    }

    case "HEAT_LOCKED": {
      // Mark all lanes in this heat official
      for (const swim of Object.values(meet.swims)) {
        if (swim.eventId === msg.eventId && swim.heat === msg.heat) {
          swim.status = "official";
        }
      }
      this.debounceSave(meetId, meet);
      return true;
    }
  }

  return false;
}

private saveTimer: any = null;
private debounceSave(meetId: string, meet: MeetManifest) {
  clearTimeout(this.saveTimer);
  this.saveTimer = setTimeout(() => {
    this.saveManifest(meetId, meet);
  }, 500); // Batch writes to disk
}
}

export const meetCache = new MeetCacheManager();

```

What This Unlocks

1. Atomic Navigation:

Navigating between Event 3 Heat 1 and Event 4 Heat 2 requires zero data fetching. Everything the route needs is already sitting right there in memory.

2. Trivial Offline Bootstrapping:

If a volunteer opens their phone cold on deck with zero cell service, the shell's `clientLoader` reads `localStorage.getItem("meet:123")` once and boots the entire app instantly.

3. Zero Sync Jitter:

When an incoming WebSocket message arrives, you mutate the exact swim in the in-memory object, call `revalidator.revalidate()`, and Remix notifies whichever active components depend on that data.

4. Simple Clean-up:

When the meet ends or the user views past archives, deleting the meet is `localStorage.removeItem("meet:123")`. No sweeping orphaned records.

User prompt: i like that recommendation, but remember we'll be getting a good amount of chatter about lane assignments, watches recorded, swims finalized... about 8 lines x 3 watches within a couple seconds when a heat finishes (plus admin messages)... and those are just to individually-keyable watches, generally...

Response: That burst at the finish of a heat—24 watches hitting the pool within a 2-to-3-second window—is the exact hot path where a naive "one big object" sync can choke if you're not careful about **when** and **how** you revalidate.

If 24 watch messages arrive over the socket in 2 seconds, and each message immediately triggers `JSON.stringify(entireMeet)` to disk and calls `revalidator.revalidate()`, you get:

1. **24 disk writes** fighting for `localStorage` locks on a hot mobile device.
2. **24 Remix revalidation cycles**, thrashing React's render scheduler right as timers are trying to tap their stop buttons.

You do **not** need to split your storage into 800 separate keys to fix this. Instead, decouple **in-memory ingestion** from **disk flushing** and **Remix revalidation** using two simple batching techniques.

1. In-Memory Micro-Patches (Instant $O(1)$ Mutation)

In JavaScript memory, mutating a single property on a normalized map takes less than a microsecond ($< 0.001\text{ms}$). Mutating 24 watches in memory takes practically 0ms of CPU time.

The issue isn't the memory update; it's the **downstream work** (re-rendering React and writing to disk).

2. Microtask Batching for revalidator.revalidate()

When 24 WebSocket messages arrive back-to-back over the socket, you don't revalidate 24 times. You collect the incoming messages in memory and queue **a single revalidation on the next animation frame or microtask**:

```
// app/lib/meetCache.ts

class MeetCacheManager {
  private manifests = new Map<string, MeetManifest>();
  private revalidateQueued = false;
  private diskSaveTimer: any = null;

  applyPatch(meetId: string, msg: any, triggerRevalidate: () => void): void {
    const meet = this.manifests.get(meetId);
    if (!meet) return;

    // 1. Direct O(1) in-memory mutation – ultra-fast (<0.005ms)
    switch (msg.type) {
      case "WATCH_RECORDED": {
        const key = `${msg.eventId}:h${msg.heat}:l${msg.lane}` as SwimKey;
        const swim = meet.swims[key];
        if (swim) {
          if (!swim.watches) swim.watches = [];
          // Upsert or push the watch
          const idx = swim.watches.findIndex((w) => w.id === msg.watch.id);
          if (idx >= 0) {
            swim.watches[idx] = msg.watch;
          } else {
            swim.watches.push(msg.watch);
          }
        }
        break;
      }
      case "SWIM_FINALIZED": {
        const key = `${msg.eventId}:h${msg.heat}:l${msg.lane}` as SwimKey;
        if (meet.swims[key]) {
          meet.swims[key].officialTimeMs = msg.officialTimeMs;
          meet.swims[key].status = "official";
        }
        break;
      }
    }

    // 2. Schedule single UI revalidation (Coalesces 24 burst events into 1 React paint)
    if (!this.revalidateQueued) {
      this.revalidateQueued = true;
      // Wait for the current event loop burst to settle before notifying Remix
      requestAnimationFrame(() => {
        this.revalidateQueued = false;
        triggerRevalidate();
      });
    }

    // 3. Debounce Disk Persistence (Don't hammer localStorage during a race finish)
    this.scheduleDiskFlush(meetId, meet);
  }

  private scheduleDiskFlush(meetId: string, meet: MeetManifest) {
    clearTimeout(this.diskSaveTimer);
    // Wait until the finish burst goes quiet for 1.5 seconds before writing full JSON to disk
    this.diskSaveTimer = setTimeout(() => {
      try {
        localStorage.setItem(`meet:${meetId}`, JSON.stringify(meet));
      } catch (_) {}
    }, 1500);
  }
}

export const meetCache = new MeetCacheManager();
```

What Happens During the Heat Finish Burst?

Here is what the execution timeline looks like when all 8 lanes finish:

Time (ms)	Event	Cache Action
0ms	Lane 3 Watch 1 touches wall	Mutates in memory. Queues rAF.
50ms	Lane 4 Watch 1 touches wall	Mutates in memory. (rAF already pending)
120ms	Lane 3 Watch 2 touches wall	Mutates in memory.
150ms	Lane 5 Watch 1 touches wall	Mutates in memory.
... (20 more watch messages arrive over ~1.5s)		
16.6ms	Next Frame (rAF tick)	Single revalidator.revalidate() runs! (Admin grid updates smoothly at 60fps)
+1500ms	Quiet on deck	Single debounced write to localStorage.

Why This Beats Granular K-V Storage

If you had split each watch into its own key (`watch:${id}`) to handle this chatter:

- You'd be calling `localStorage.setItem()` 24 times in 2 seconds. In Safari on iOS, 24 rapid, synchronous `localStorage` writes will cause UI thread lockups and dropped frames on your stopwatch touch buttons.
- Any admin view or coach view showing the whole heat would have to re-render 24 times as each discrete key changed.

With In-Memory Mutations + Frame-Throttled Revalidation + Debounced Disk Writes:

- **The Timer Phone:** Tapping the stop button is silky smooth because JavaScript memory writes take 0 CPU overhead.
- **The Admin Screen:** The live grid repaints at a clean 60 frames per second instead of jittering on every millisecond socket arrival.
- **Storage:** Disk I/O happens in one clean write after the heat is done and the swimmers are clearing the pool.

User prompt: You mentioned passing 250 KB of json takes 1 to 3 ms... how are we avoiding doing that every time we get a patch or update? Is it because we're keeping the deserialized version in memory? You say JavaScript memory writes take 0ms but we'd have to parse and find keys and then updates would be fast, right?

Response: Yes, exactly—you hit the nail on the head.

The entire reason this is fast is because **you only deserialize the 250 KB payload once**, when the app first loads. After that initial paint, the full meet lives continuously in the JavaScript engine's heap memory as live, native JavaScript objects.

Here is the exact breakdown of the difference between handling the incoming patch versus handling the full document.

What Actually Arrives Over the WebSocket

When Lane 4's watch clicks the wall, the server is **not** re-sending the 250 KB meet manifest. It is sending a tiny, targeted packet:

```
{"type":"WATCH_RECORDED","eventId":"ev_101","heat":2,"lane":4,"watch":{"id":"w_829","timeMs":24820}}
```

- **Packet size:** $\approx$ 100 bytes (not 250 KB).
- **JSON.parse(msg.data) cost:** Less than 0.005 ms (a few microseconds).

Why Finding the Key and Updating Takes Virtually 0ms

Once that 100-byte message is parsed into an object, finding the right place in your live in-memory meet does not require scanning arrays or looping through objects.

Because swims is a normalized dictionary keyed by slot:

```
// 1. Compute string key: <0.001 ms
const key = `${msg.eventId}:h${msg.heat}:l${msg.lane}`;

// 2. Hash-table lookup in JS engine heap: <0.001 ms
const swim = meet.swims[key];

// 3. Mutate property in memory: <0.001 ms
swim.watches.push(msg.watch);
```

Under the hood in V8 / JavaScriptCore:

1. **Property Lookup:** `meet.swims[key]` is an internal hash table lookup. It finds the memory pointer to that exact `Swim` object immediately ($O(1)$).
2. **Direct Mutation:** `swim.watches.push(...)` appends a reference to that object's existing array in RAM.
3. **Zero Cloning / Serialization:** There is no `JSON.stringify()`, no `JSON.parse()` of the big meet, and no deep object cloning happening in that path.

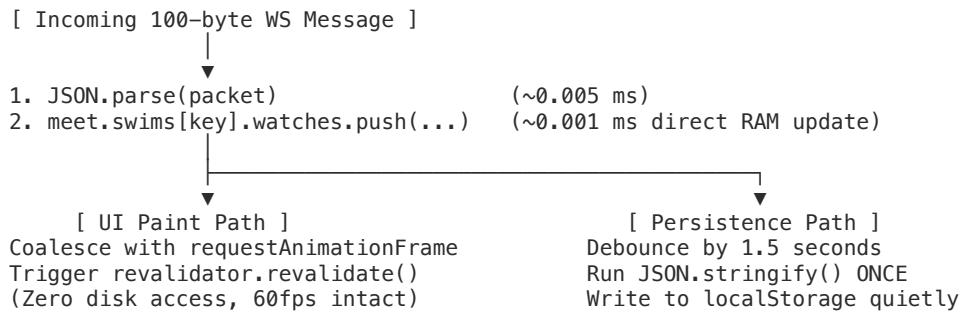
Where the Trap Would Be (And What We Avoid)

The trap—which many apps fall into—is doing this:

```
// ❌ THE SLOW WAY: Treating storage as state
ws.onmessage = (event) => {
  // 1. Read 250 KB from disk
  const raw = localStorage.getItem(`meet:${meetId}`);
  // 2. Parse 250 KB JSON (takes 2-3ms)
  const meet = JSON.parse(raw);
  // 3. Mutate
  meet.swims[key].watches.push(msg.watch);
  // 4. Serialize 250 KB back to text (takes 3-5ms)
  localStorage.setItem(`meet:${meetId}`, JSON.stringify(meet));
};
```

If you do *that* 24 times during a heat finish, you will burn ~150ms of CPU time, lock the storage thread, and drop touch frames on the stopwatch.

The Clean Architecture: Memory First, Storage Trailing



The deserialized tree sits warm in memory for the entire duration of the meet. Updates modify pointers in RAM at microsecond speeds, and the disk is only updated during quiet moments between races.

User prompt: could you reproduce exactly what you have in mind at this point for meetcache.ts?

Response: Here is the complete, self-contained meetCache.ts implementation incorporating all the structural decisions: the composite SwimKey mapping, in-memory live tree mutations, microtask-batched revalidations, and debounced local storage persistence.

```
// app/lib/meetCache.ts

export type SwimKey = `${string}:h${number}:l${number}`;

export interface SwimSlot {
  eventId: string;
  heat: number;
  lane: number;
}

export function toSwimKey(slot: SwimSlot): SwimKey {
  return `${slot.eventId}:h${slot.heat}:l${slot.lane}`;
}

export function parseSwimKey(key: SwimKey): SwimSlot {
  const [eventId, h, l] = key.split(":");
  return {
    eventId,
    heat: Number(h.replace("h", "")),
    lane: Number(l.replace("l", "")),
  };
}

export interface Watch {
  id: string;
  swimId: string;
  submittedBy: string;
  userId?: string;
  role: "timer" | "chief_timer" | "admin";
  slot: number;
  timeMs?: number;
}
```

```

    startedAt?: number;
    stoppedAt?: number;
    submittedAt: number;
  }

export interface Swim {
  id: string;
  eventId: string;
  heat: number;
  lane: number;
  athleteId?: string;
  athleteName?: string;
  athleteTeam?: string;
  exhibition: boolean;
  status: "pending" | "official" | "dq" | "dns";
  officialTimeMs?: number;
  decidedAt?: number;
  decidedBy?: string;
  watches?: Watch[];
}

export interface MeetEvent {
  id: string;
  position: number;
  eventNumber: number;
  distance: number;
  stroke: "free" | "back" | "breast" | "fly" | "im" | "medley";
  gender: "boys" | "girls" | "mixed";
  name?: string;
  totalHeats: number;
}

export interface MeetManifest {
  id: string;
  name: string;
  isLive: boolean;
  currentEventId?: string;
  currentHeatNumber?: number;
  events: Record<string, MeetEvent>;
  swims: Record<SwimKey, Swim>;
}

export type LiveSocketMessage =
  | {
    type: "WATCH_RECORDED";
    eventId: string;
    heat: number;
    lane: number;
    watch: Watch;
  }
  | {
    type: "SWIM_STATUS_UPDATED";
    eventId: string;
    heat: number;
    lane: number;
    status: "pending" | "official" | "dq" | "dns";
    officialTimeMs?: number;
    decidedAt?: number;
    decidedBy?: string;
  }
  | {
    type: "LANE_ASSIGNMENT_CHANGED";
    eventId: string;
    heat: number;
    lane: number;
    athleteId?: string;
    athleteName?: string;
    athleteTeam?: string;
    exhibition?: boolean;
  }
  | {
    type: "ACTIVE_DECK_POSITION";
    eventId: string;
    heatNumber: number;
  }

```

```

    }
    | {
      type: "EVENT_RESEEDDED";
      eventId: string;
      totalHeats: number;
      swims: Swim[];
    };
  };

class MeetCacheManager {
  // In-memory heap cache of deserialized manifests
  private manifests = new Map<string, MeetManifest>();

  // Coalescing queue for high-frequency bursts
  private isRevalidationPending = false;
  private diskSaveTimers = new Map<string, ReturnType<typeof setTimeout>>();

  /**
   * Retrieves the manifest synchronously from memory if warm,
   * falling back to localStorage during initial bootstrap.
   */
  getManifest(meetId: string): MeetManifest | null {
    if (this.manifests.has(meetId)) {
      return this.manifests.get(meetId)!;
    }

    if (typeof window !== "undefined") {
      try {
        const raw = localStorage.getItem(`meet:${meetId}`);
        if (raw) {
          const parsed = JSON.parse(raw) as MeetManifest;
          this.manifests.set(meetId, parsed);
          return parsed;
        }
      } catch (err) {
        console.warn("Failed to load meet manifest from storage", err);
      }
    }

    return null;
  }

  /**
   * Stores fresh manifest from initial SSR/loader hydration in RAM and disk.
   */
  saveManifest(meetId: string, manifest: MeetManifest): void {
    this.manifests.set(meetId, manifest);

    if (typeof window !== "undefined") {
      try {
        localStorage.setItem(`meet:${meetId}`, JSON.stringify(manifest));
      } catch (err) {
        console.warn("Failed to persist meet manifest to storage", err);
      }
    }
  }

  /**
   * Applies incoming socket patches directly to the in-memory object graph in O(1) time.
   * Batches downstream Remix revalidation to next animation frame and debounces disk writes.
   */
  applyPatch(
    meetId: string,
    msg: LiveSocketMessage,
    onRevalidate: () => void
  ): boolean {
    const meet = this.manifests.get(meetId);
    if (!meet) return false;

    let didMutate = false;

    switch (msg.type) {
      case "WATCH_RECORDED": {
        const key = toSwimKey({ eventId: msg.eventId, heat: msg.heat, lane: msg.lane });
        const swim = meet.swims[key];

```

```

    if (swim) {
      if (!swim.watches) swim.watches = [];
      const existingIdx = swim.watches.findIndex((w) => w.id === msg.watch.id);
      if (existingIdx >= 0) {
        swim.watches[existingIdx] = msg.watch;
      } else {
        swim.watches.push(msg.watch);
      }
      didMutate = true;
    }
    break;
  }

  case "SWIM_STATUS_UPDATED": {
    const key = toSwimKey({ eventId: msg.eventId, heat: msg.heat, lane: msg.lane });
    const swim = meet.swims[key];
    if (swim) {
      swim.status = msg.status;
      if (msg.officialTimeMs !== undefined) swim.officialTimeMs = msg.officialTimeMs;
      if (msg.decidedAt !== undefined) swim.decidedAt = msg.decidedAt;
      if (msg.decidedBy !== undefined) swim.decidedBy = msg.decidedBy;
      didMutate = true;
    }
    break;
  }

  case "LANE_ASSIGNMENT_CHANGED": {
    const key = toSwimKey({ eventId: msg.eventId, heat: msg.heat, lane: msg.lane });
    const swim = meet.swims[key];
    if (swim) {
      swim.athleteId = msg.athleteId;
      if (msg.athleteName !== undefined) swim.athleteName = msg.athleteName;
      if (msg.athleteTeam !== undefined) swim.athleteTeam = msg.athleteTeam;
      if (msg.exhibition !== undefined) swim.exhibition = msg.exhibition;
      didMutate = true;
    }
    break;
  }

  case "ACTIVE_DECK_POSITION": {
    meet.currentEventId = msg.eventId;
    meet.currentHeatNumber = msg.heatNumber;
    didMutate = true;
    break;
  }

  case "EVENT_RESEED": {
    if (meet.events[msg.eventId]) {
      meet.events[msg.eventId].totalHeats = msg.totalHeats;
    }
    // Remove prior swims for this event
    for (const key of Object.keys(meet.swims) as SwimKey[]) {
      if (parseSwimKey(key).eventId === msg.eventId) {
        delete meet.swims[key];
      }
    }
    // Populate newly seeded swims
    for (const swim of msg.swims) {
      const key = toSwimKey({ eventId: swim.eventId, heat: swim.heat, lane: swim.lane });
      meet.swims[key] = swim;
    }
    didMutate = true;
    break;
  }
}

if (!didMutate) return false;

// 1. Coalesce UI revalidations so rapid multi-timer bursts only repaint once per frame
if (!this.isRevalidationPending) {
  this.isRevalidationPending = true;
  if (typeof window !== "undefined" && "requestAnimationFrame" in window) {
    requestAnimationFrame(() => {
      this.isRevalidationPending = false;
    });
  }
}

```

```

        onRevalidate();
    });
} else {
    queueMicrotask(() => {
        this.isRevalidationPending = false;
        onRevalidate();
    });
}
}

// 2. Debounce serialization & disk write until pool action settles
this.scheduleDiskPersist(meetId, meet);

return true;
}

private scheduleDiskPersist(meetId: string, meet: MeetManifest): void {
    if (typeof window === "undefined") return;

    const existingTimer = this.diskSaveTimers.get(meetId);
    if (existingTimer) clearTimeout(existingTimer);

    const timer = setTimeout(() => {
        try {
            localStorage.setItem(`meet:${meetId}`, JSON.stringify(meet));
        } catch (err) {
            console.warn("Failed debounced write to localStorage", err);
        } finally {
            this.diskSaveTimers.delete(meetId);
        }
    }, 1200); // 1.2s debounce buffer

    this.diskSaveTimers.set(meetId, timer);
}

clear(meetId?: string): void {
    if (meetId) {
        this.manifests.delete(meetId);
        const timer = this.diskSaveTimers.get(meetId);
        if (timer) clearTimeout(timer);
        this.diskSaveTimers.delete(meetId);
        if (typeof window !== "undefined") {
            localStorage.removeItem(`meet:${meetId}`);
        }
    } else {
        this.manifests.clear();
        for (const timer of this.diskSaveTimers.values()) {
            clearTimeout(timer);
        }
        this.diskSaveTimers.clear();
    }
}
}

export const meetCache = new MeetCacheManager();

```

User prompt: *hmm...wondering...for MeetManifest, should 'watches' be its own top level entry on 'meet', like 'swims'? export type WatchKey = `e\${string}:h\${number}:l\${number}:user\${string}:slot\${number}`; export interface MeetManifest { id: string; name: string; isLive: boolean; currentEventId?: string; currentHeatNumber?: number; events: Record<string, Event>; swims: Record<SwimKey, Swim>; watches: Record<WatchKey, Watch>; } would mirror our DB (making building of the meetmanifest on the server cleaner), make getting/adding/updating watches (key for the timer and admin UIs) quick and easy.*

Response: Yes, elevating watches to a top-level dictionary on MeetManifest is a noticeably cleaner architecture.

It completely removes the last bit of nested array mutation in meetCache, aligns your client manifest directly with your Durable Object's SQLite tables, and gives you idempotent, $O(1)$ watch updates.

Here is an analysis of why this works better, along with a key refinement for the WatchKey structure.

Why Top-Level watches Wins

1. Ingestion Becomes a Pure Key-Value Put

When watches lived inside `swim.watches[]`, updating a watch required finding the swim, checking if the array existed, running `.findIndex()` to see if this watch already existed, and either updating or `.push()`ing.

With top-level watches:

```
// Incoming WebSocket message
meet.watches[msg.watchKey] = msg.watch;
```

It is a single, direct, atomic assignment. If an offline queue flushes and a duplicate watch packet arrives, it simply writes the identical object over the same key. Zero deduplication logic needed.

2. Mirroring the SQLite Ingestion Pipeline

In your Durable Object, your SQLite tables are:

- events
- swims
- watches

When the DO builds the initial manifest payload for SSR / `clientLoader`:

```
const [events, swims, watches] = await Promise.all([
  db.select().from(eventsTable).all(),
  db.select().from(swimsTable).all(),
  db.select().from(watchesTable).all(),
]);

return {
  id: meetId,
  events: Object.fromEntries(events.map(e => [e.id, e])),
  swims: Object.fromEntries(swims.map(s => [toSwimKey(s), s])),
  watches: Object.fromEntries(watches.map(w => [toWatchKey(w), w])),
};
```

You don't have to write server-side grouping queries or stitch child watches arrays onto parent swims before shipping the JSON across the wire.

3. Zero Ambiguity Around Deletions or Discards

If an admin discards a bad watch (e.g., volunteer bumped their phone in their pocket), deleting it is simply:

```
delete meet.watches[watchKey];
```

No array splicing or filter passes required.

The Crucial Refinement: The WatchKey Format

Look closely at your proposed key:

```
`e${string}:h${number}:l${number}:users${string}:slot${number}`
```

Including **both user and slot** creates an identity hazard on a high school pool deck:

- If Timer 1 (User A) is assigned to Lane 3 as "Watch 1", their key is `...:userA:slot1`.
- What happens if Timer 1's phone battery dies, and another volunteer (User B) takes over Watch 1 for that lane?
- If User B submits for Watch 1, their key is `...:userB:slot1`.
- Now you have **two active records for Slot 1 in the same lane**.

The Golden Rule of Pool Timing Keys: Identity by Slot vs. Identity by Device

In swim meet operations, you care about either:

1. **The Physical Slot (Standard Meet Ops):** There are at most 3 official watch slots per lane (Watch 1, Watch 2, Watch 3).
2. **The Device / Submission ID (Pure Audit Log):** Every device submission is unique.

Because you have multiple timers per lane, you want **the physical slot** to be the unique identifier for the swim, with `submittedBy / userId` as properties on the record.

Recommended WatchKey:

```
// Keyed strictly by physical race position + watch slot (1, 2, or 3)
export type WatchKey = `${string}:h${number}:l${number}:s${number}`;
```

```
export interface WatchSlotKey {
  eventId: string;
  heat: number;
  lane: number;
  slot: number; // 1, 2, or 3
}

export function toWatchKey(s: WatchSlotKey): WatchKey {
  return `${s.eventId}:h${s.heat}:l${s.lane}:s${s.slot}`;
}
```

Why this key structure is bulletproof:

- Heat 2, Lane 4 has exactly three possible watch slots:
 - ev_101:h2:l4:s1
 - ev_101:h2:l4:s2
 - ev_101:h2:l4:s3
- If a timer resubmits, or a second timer corrects a time for Slot 1, it cleanly replaces Slot 1 without creating phantom 4th and 5th watches for that lane.

How the UI Consumes This

Now both the Timer and Admin UIs get clean, direct access without looping:

1. The Timer Stopwatch View (Wants *their* specific watch)

```
// The volunteer knows they are timing Lane 4, Slot 1:
const watchKey = toWatchKey({ eventId, heat, lane, slot: 1 });
const myWatch = meet.watches[watchKey];
```

2. The Admin Heat Reconciliation Grid (Wants all 1–3 watches for Lane 4)

Because the slot numbers are bounded (1 to 3), the admin grid doesn't even need an array filter:

```
function getLaneWatches(meet: MeetManifest, eventId: string, heat: number, lane: number): (Watch | undefined) {
  return [
    meet.watches[toWatchKey({ eventId, heat, lane, slot: 1 })],
    meet.watches[toWatchKey({ eventId, heat, lane, slot: 2 })],
    meet.watches[toWatchKey({ eventId, heat, lane, slot: 3 })],
  ];
}
```

The Final MeetManifest Shape

```
export interface MeetManifest {
  id: string;
  name: string;
  isLive: boolean;
  currentEventId?: string;
  currentHeatNumber?: number;
  events: Record<string, MeetEvent>; // Keyed by eventId
  swims: Record<SwimKey, Swim>; // Keyed by `${eventId}:h${heat}:l${lane}`
  watches: Record<WatchKey, Watch>; // Keyed by `${eventId}:h${heat}:l${lane}:s${slot}`
}
```

This makes MeetManifest a completely flat, normalized database cache in client memory. All three collections (events, swims, watches) have deterministic $O(1)$ access, zero nested arrays, and line up 1:1 with your SQLite tables.

User prompt: that slot idea would be much cleaner, except in this app, times can also be collected by coaches using the multi-laner timer and a time can be manually typed by an admin for a swim... i was representing all using the same watch data structure (which does make sense), but it means there can more than just three slots ... timer1, timer2, timer3, admin, coach (or maybe even coach1, coach2... maybe more?).

Response: That changes the game. If coaches are taking unofficial splits across multiple lanes and admins can punch in a manual time, you are dealing with two completely different categories of data:

1. **Official Lane Timing Inputs** (Volunteer timers on the blocks → feeds the official result algorithm).
2. **Supplemental / Role-Specific Inputs** (Coach splits, admin manual overrides, chief timer backups).

Conflating an unofficial coach split or an admin manual override into the same slot space as lane timers creates scoring hazards and identity clashes.

The Two Ways to Model This Cleanly

Approach 1: Semantic Source Keys (Cleanest if sources are bounded)

Instead of a generic integer slot (1, 2, 3), key the watch by its **functional source / role**:

```
export type WatchSource =
  | `t${1 | 2 | 3}`           // Official Lane Timers: 't1', 't2', 't3'
  | "admin"                   // Table / Admin Manual Entry
  | `coach:${string}`;        // Coach split: 'coach:coachBob' or 'coach:teamA'

export type WatchKey = `${string}:h${number}:l${number}:${WatchSource}`;
```

Examples of generated keys:

- `ev_101:h2:l4:t1` → Official Timer 1 for Lane 4
- `ev_101:h2:l4:t2` → Official Timer 2 for Lane 4
- `ev_101:h2:l4:admin` → Admin table override / manual paper transcription
- `ev_101:h2:l4:coach:usr_coach_mike` → Coach Mike's multi-lane watch split

Why this is great:

- **Zero Collision Risk:** Coach Mike's multi-lane watch will never overwrite Timer 1's watch or the Admin's override.
- **Trivial Scoring Rules:** When calculating the official time, your math engine simply queries `key.endsWith(':t1'), :t2, :t3` (or checks `watch.role === 'timer'`). It never accidentally averages a coach's unofficial hand-time into a state-qualifying official result.
- **Deterministic $O(1)$ lookups:**
 - Volunteer Timer 2 loads `/timer/.../lane/4/slot/2` → directly reads `...:t2`.
 - Admin types in a paper time → writes directly to `...:admin`.

Approach 2: Global watchId with an Index Map (Maximum Flexibility)

If you have an arbitrary number of coaches, parents, or chief timers clicking watches and you don't want bounded keys at all:

Don't try to cram event/heat/lane into the WatchKey. Key watches simply by their unique ID (`watch.id` / UUID), and maintain a fast index:

```
export interface MeetManifest {
  events: Record<string, MeetEvent>;
  swims: Record<SwimKey, Swim>;
  watches: Record<string, Watch>; // Keyed directly by watch.id (UUID)
}
```

To look up watches for a swim without filtering all watches in the meet, you can either:

1. Keep a lightweight string array of watch IDs on the Swim:

```
export interface Swim {
  id: string;
  eventId: string;
  heat: number;
  lane: number;
  watchIds?: string[]; // ['w_123', 'w_456']
  // ...
}
```

- An incoming watch socket packet does two fast updates:

```
meet.watches[msg.watch.id] = msg.watch;
if (!swim.watchIds.includes(msg.watch.id)) {
  swim.watchIds.push(msg.watch.id);
}
```

- When looking up Lane 4's times:

```
const swim = meet.swims[swimKey];
const laneWatches = swim.watchIds?.map(id => meet.watches[id]) ?? [];
```

Which Approach Fits Better?

- **Use Approach 1 (Semantic Source Key: :t1, :t2, :admin, :coach: ...)** if your UI explicitly assigns volunteers to physical slots (e.g. "You are Timer 1 on Lane 4") and you want **guaranteed idempotency without maintaining an index array**. It keeps the database completely flat and makes distinguishing official times from coach splits trivial.
- **Use Approach 2 (UUID Key + watchIds[] on Swim)** if timing is completely open-ended and unconstrained, where 4 coaches and 3 timers might all click the same lane simultaneously without designated roles.

For high school dual meets, **Approach 1** usually wins because the rulebook strictly delineates between the 3 official lane watches, the admin scorer's table, and coach training splits.

User prompt: very nice - i'll go with approach to, still simple and keeps things flexible. now, related data question - i probably also need athletes hung off MeetManifest, no?

Response: Yes, absolutely. Having a top-level athletes dictionary on MeetManifest is essential once you move from passive timing to active deck management.

```
export interface Athlete {
  id: string;
  firstName: string;
  lastName: string;
  teamId: string;
  teamName?: string;
  gender: "boys" | "girls";
  grade?: number; // 9, 10, 11, 12
  usaSwimmingId?: string;
}

export interface MeetManifest {
  id: string;
  name: string;
  isLive: boolean;
  currentEventId?: string;
  currentHeatNumber?: number;
  events: Record<string, MeetEvent>;
  swims: Record<SwimKey, Swim>;
  watches: Record<string, Watch>;
  athletes: Record<string, Athlete>; // Keyed by athlete.id
}
```

Why Deck Operations Need athletes in Memory

1. Instant Walk-Up and Swap Autocomplete (Offline-Ready)

When a coach runs up to the table or a timer has an empty lane and says, "Hey, Marcus is swimming in Lane 5 instead," you cannot make a round-trip network call to search the database. With athletes in memory:

- The timer or admin types "Mar..." into the lane modal.
- The UI instantly filters the local roster:

```
const options = Object.values(meet.athletes).filter(
  (a) => a.teamId === currentTeam && a.gender === eventGender
);
```

- Tapping Marcus updates the lane assignment with zero latency, even without an active internet connection.

2. Enforcing Entry Limits On Deck

High school dual meet rules are strict (e.g., maximum 4 events total, maximum 2 individual events per athlete). If a coach tries to slip an athlete into an extra exhibition race, you can calculate their current entry count immediately from the client cache:

```
export function getAthleteSwimsCount(meet: MeetManifest, athleteId: string) {
  return Object.values(meet.swims).filter(
    (s) => s.athleteId === athleteId && s.status !== "dns"
  ).length;
}
```

3. Relays Without Duplicate Submissions

For relays, the parent Swim record represents the four-swimmer relay team. Instead of cramming 4 full athlete sub-objects into the swim, you store an array of IDs:

```
relayAthleteIds: [string, string, string, string];
```

Your UI resolves the four names via `meet.athletes[id]` cleanly without bloating the swim record.

Keep Snapshot Fields on Swim as Backups

Even with `athletes: Record<string, Athlete>` at the root, you should still keep `athleteName` and `athleteTeam` on the `Swim` entity:

```
export interface Swim {
  id: string;
  eventId: string;
  heat: number;
  lane: number;
  athleteId?: string;
  athleteName?: string; // Fallback snapshot
  athleteTeam?: string; // Fallback snapshot
  // ...
}
```

Why keep both?

- **Known Roster Athlete:** When selected, set `athleteId: athlete.id`, `athleteName: "Marcus Vance"`, and `athleteTeam: "Chaparral"`. Lookups join against `meet.athletes[athleteId]`.
- **Guest / Unregistered Swimmer:** If an opposing team brought a swimmer who wasn't on the pre-meet import file, a timer can simply type "John Doe" into the field. You leave `athleteId: undefined` and write directly to `athleteName`.

This keeps deck operations unblocked: official roster members stay fully relational and auditable, while last-second guest swimmers can still be timed and scored without first navigating to an "Add Athlete" admin screen.

User prompt: nice! next data model question - on my MeetManifest object, which is better for representing entries (our planned athlete/event pairings): `entries: Record<string, Entry>`; or `entries: Record<string, Entry[]>`; in the first, the string is the Entry id... in the second, it's the eventId, and would return the array of entries for this event... or is there something else you'd suggest?

Response: Neither of those is ideal once you are operating on a live pool deck.

The cleanest, most resilient representation is a flat map keyed by the natural composite identity of an entry: **athleteId + eventId**.

```
export type EntryKey = `${string}:${string}`; // `${eventId}:${athleteId}`

export function toEntryKey(eventId: string, athleteId: string): EntryKey {
  return `${eventId}:${athleteId}`;
}

export interface MeetManifest {
  // ...
  entries: Record<EntryKey, Entry>;
}
```

Why the Other Two Options Break Down

Option 1: Record<string, Entry> (Keyed by entry.id)

- **The Problem:** It forces an unnatural ID lookup when you almost never have an `entry.id` in hand during meet ops.
- **Deck Reality:** When a timer or admin looks at Lane 4, they know the `eventId` and the `athleteId`. If entries are keyed by arbitrary UUIDs (`entries["ent_9a8f2..."]`), you have to do an expensive array scan:

```
Object.values(meet.entries).find(e => e.eventId === currentEvent && e.athleteId === currentAthlete);
```

That destroys the $O(1)$ performance advantage of a normalized map.

Option 2: Record<string, Entry[]> (Keyed by eventId returning an array)

- **The Problem:** Nested mutable arrays in an event-driven system.
- **Deck Reality:**
 - If a coach scratches an athlete or changes a seed time over WebSockets, updating a nested array requires finding the index, splicing, or mapping the array.
 - If an entry update packet arrives concurrently, you risk race conditions or duplicate entries in that array.
 - Checking if an athlete is already entered in an event still requires an inner `.find()`.

Why Record<eventId :{athleteId}, Entry> Wins

In swimming rules, **an athlete can only be entered once in a given event**. Therefore, (eventId, athleteId) is a natural, immutable primary key.

1. Instant $O(1)$ Seed Time & Exhibition Checks

When hydrating a Swim row or rendering a psych sheet comparison on deck:

```
const entryKey = toEntryKey(swim.eventId, swim.athleteId);
const entry = meet.entries[entryKey];

const seedTime = entry?.seedTimeMs ?? null;
const isExhibition = entry?.isExhibition ?? false;
```

Zero array searching, zero loop overhead.

2. Atomic WebSocket Patches

If a coach updates a seed time or scratches an entry before the meet starts:

```
// Incoming patch:
const key = toEntryKey(msg.eventId, msg.athleteId);

if (msg.type === "ENTRY_SCRATCHED") {
  delete meet.entries[key];
} else if (msg.type === "ENTRY_UPDATED") {
  meet.entries[key] = msg.entry;
}
```

Pure, idempotent dictionary mutations without touching nested arrays.

3. Listing Entries for an Event or Athlete is Trivial

Whenever you do need a filtered list (e.g., viewing an event's entry list or calculating an athlete's total events for entry limits):

```
// Get all entries for an event (sorted by seed time)
export function getEventEntries(meet: MeetManifest, eventId: string): Entry[] {
  return Object.values(meet.entries)
    .filter(e => e.eventId === eventId)
    .sort((a, b) => (a.seedTimeMs ?? Infinity) - (b.seedTimeMs ?? Infinity));
}

// Check an athlete's current entry count against the dual-meet 4-event limit
export function getAthleteEntryCount(meet: MeetManifest, athleteId: string): number {
  return Object.values(meet.entries).filter(e => e.athleteId === athleteId).length;
}
```

The Recommended Entry Shape

```
export interface Entry {
  id: string; // Database PK for relational sync
  eventId: string;
  athleteId: string;
  seedTimeMs?: number;
  isExhibition: boolean;
  submittedByTeamId: string;
}

export type EntryKey = `${string}:${string}`; // `${eventId}:${athleteId}`
```

Keeping entries as Record<EntryKey, Entry> maintains the exact same flat, predictable design pattern established for swims ({eventId}:h{heat}:l{lane}) and keeps your entire client manifest uniform.

User prompt: beautiful - thank you! going back to SwimKey - should that be eventId|heat|lane or eventNumber|heat|lane ? i like event number since it's always unique within a meet (though not necessarily globally) and it's what humans think in terms of... but all the other entry/swim/event related data keys off eventId (and, creating a SwimKey then requires getting eventNumber off the event object, instead of just using the eventId its referenced by)

Response: Stick with **eventId** for SwimKey:

```
export type SwimKey = `${string}:h{number}:l{number}`; // `${eventId}:h{heat}:l{lane}`
```

While `eventNumber` is how humans communicate on deck, using it as an internal key creates several architectural and practical headaches:

1. Eliminates Unnecessary Object Lookups

Every `Swim`, `Entry`, and incoming `WebSocket` payload already carries `eventId`.

- **With `eventId`:** Generating a `SwimKey` is a direct, zero-cost string template:

```
const key = `${swim.eventId}:h${swim.heat}:l${swim.lane}`;
```

- **With `eventNumber`:** To form or parse a key, you are forced to dereference the parent event every time:

```
const event = meet.events[swim.eventId];
const key = `ev${event.eventNumber}:h${swim.heat}:l${swim.lane}`;
```

If an incoming socket packet or background worker doesn't have the hydrated event object immediately in scope, it can't even compute the key to find the swim in cache.

2. High School Event Numbering Isn't Always a Clean Unique Integer

Relying on `eventNumber` as a database or dictionary key breaks when standard dual-meet numbering realities show up:

- **Combined Heats / Split Scoring:** In many dual meets, coaches combine the Girls 500 Free (Event 7) and Boys 500 Free (Event 8) into a single heat to save time. If swimmers in the same physical heat are tied to different event numbers, an `eventNumber`-based key gets awkward.
- **Suffixes & Exhibition Events:** High school schedules often feature non-standard numbers like Event 3A (JV or Unified heats) or Event 101. If `eventNumber` is an integer, you lose flexibility; if it's a string, you invite casing and typo mismatches.
- **Reordering Events:** If the deck referee decides to run the 100 Breaststroke earlier because a swimmer has to catch an academic bus, changing `eventNumber` would mutate every single `SwimKey` across that event, breaking any active in-flight URLs or form queues.

3. URLs and Consistency Across the App

Route parameters and relational keys stay uniform across your entire stack:

- `meet.entries` is keyed by `${eventId}:${athleteId}`
- `meet.swims` is keyed by `${eventId}:h${heat}:l${lane}`
- Route params use `$event` → matching `event.id`

Whenever a human-readable label is needed for the screen, let the presentation layer do what it does best:

```
const event = meet.events[swim.eventId];
// Display: "Event 3 • Heat 1 • Lane 4"
```

Keep internal cache keys bound to immutable system IDs (`eventId`); leave `eventNumber` as an editable, display-only property on the `Event` entity.

User prompt: Is it even worth having ID? On the entry object then? I mean even in the database? Really, athlete, ID and ID serve as a compound unique ID

Response: **No, it is not worth having a surrogate id on the Entry table.**

Using a compound primary key of (`event_id`, `athlete_id`) directly in your database is cleaner, faster, and eliminates redundant state.

Why a Composite Primary Key Wins in SQLite / Durable Objects

In relational design, surrogate UUID keys (`id TEXT PRIMARY KEY`) are often added out of habit. But an entry is fundamentally an **associative entity** (a many-to-many join with attributes) connecting an athlete to an event.

Here is why dropping `id` is the right move:

1. Native Database-Level Guarantee

When you define the composite primary key:

```
CREATE TABLE entries (
  event_id TEXT NOT NULL,
  athlete_id TEXT NOT NULL,
  seed_time_ms INTEGER,
  is_exhibition INTEGER NOT NULL DEFAULT 0,
  submitted_by_team_id TEXT NOT NULL,
```

```
PRIMARY KEY (event_id, athlete_id)
);
```

SQLite automatically creates an index on (event_id, athlete_id). You get the uniqueness guarantee for free without creating a secondary UNIQUE constraint alongside an unused id index.

2. Native UPSERT Simplicity

When coaches submit lineups or tweak seed times, you don't need to query for an existing id first. SQLite handles it natively:

```
INSERT INTO entries (event_id, athlete_id, seed_time_ms, is_exhibition, submitted_by_team_id)
VALUES (?, ?, ?, ?, ?)
ON CONFLICT (event_id, athlete_id) DO UPDATE SET
  seed_time_ms = excluded.seed_time_ms,
  is_exhibition = excluded.is_exhibition;
```

3. Zero Sync Impedance Mismatch

Earlier we established that on the client, meet.entries is keyed by:

```
`${eventId}:${athleteId}`
```

If your SQL table has an arbitrary id: "ent_abc123", the server has to serialize a field that the client immediately discards or ignores.

Without id, your client model and server row align 1:1:

```
export interface Entry {
  eventId: string;
  athleteId: string;
  seedTimeMs?: number;
  isExhibition: boolean;
  submittedByTeamId: string;
}
```

When Would an id Actually Be Needed?

A surrogate id is only useful if:

1. **Child records point to it:** Earlier, we confirmed that Swim should **not** reference entry_id (it points directly to athlete_id and position), so no table holds a foreign key to entries.
2. **Multiple entries per event are allowed:** In swim rules, an athlete can never be entered twice in the same individual event.

Because neither applies, an arbitrary UUID on Entry is pure deadweight. Drop the column in both SQL and TypeScript.

User prompt: Same for swim? Drop id and use our SwimKey fields eventId, heat, lane?

Response: **Yes, drop id on Swim as well.**

Using a composite primary key of (event_id, heat, lane) in SQL and TypeScript gives you the exact same benefits you just gained on Entry: zero surrogate overhead, native uniqueness, and complete alignment between client dictionary keys and database rows.

Why a Composite Key Works for Swim

On a pool deck, the physical slot in the water **is** the natural primary key:

- Can there ever be two different swims occurring simultaneously in Event 3, Heat 2, Lane 4? No.
- Can a heat have multiple Lane 4s? No.

The physical coordinates (event_id, heat, lane) uniquely describe the slot.

```
CREATE TABLE swims (
  event_id TEXT NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  heat INTEGER NOT NULL,
  lane INTEGER NOT NULL,
  athlete_id TEXT,
  athlete_name TEXT,
  athlete_team TEXT,
  exhibition INTEGER NOT NULL DEFAULT 0,
  status TEXT NOT NULL DEFAULT 'pending',
  official_time_ms INTEGER,
  decided_at INTEGER,
  decided_by TEXT,
```

```
PRIMARY KEY (event_id, heat, lane)
);
```

The Foreign Key Cascades Cleanly into watches

If Swim doesn't have an id, what does the watches table reference?

It simply uses the composite key as its compound foreign key:

```
CREATE TABLE watches (
  id TEXT PRIMARY KEY,
  event_id TEXT NOT NULL,
  heat INTEGER NOT NULL,
  lane INTEGER NOT NULL,
  submitted_by TEXT NOT NULL,
  user_id TEXT,
  role TEXT NOT NULL DEFAULT 'timer',
  slot INTEGER NOT NULL DEFAULT 1,
  time_ms INTEGER,
  started_at INTEGER,
  stopped_at INTEGER,
  submitted_at INTEGER NOT NULL,
  FOREIGN KEY (event_id, heat, lane)
    REFERENCES swims(event_id, heat, lane)
    ON DELETE CASCADE
);
```

Why This Is a Win for Timers and the Admin Grid:

With a composite foreign key on watches:

1. **Zero Join Querying:** When a timer's phone submits a watch time for Event 2, Heat 1, Lane 4, the server **does not have to run a SELECT id FROM swims WHERE...** first. It inserts directly into watches using the route params already in the URL.
2. **Instant Compound Filtering:** Querying all watches for an entire heat doesn't require joining through a swims bridge table:

```
SELECT * FROM watches WHERE event_id = ? AND heat = ?;
```

Total Symmetry Across Your Entire Stack

Dropping surrogate IDs on both Entry and Swim brings clean symmetry to the architecture:

Entity	Primary Key (SQL)	Client Dictionary Key	Natural Identity
Event	id	event.id	Single named race entity
Entry	(event_id, athlete_id)	\${eventId}:\${athleteId}	Athlete × Event declaration
Swim	(event_id, heat, lane)	\${eventId}:h\${heat}:l\${lane}	The physical slot in the water
Watch	id (UUID)	watch.id	Specific stopwatch submission

The Simplified Swim TypeScript Interface

```
export interface Swim {
  eventId: string;
  heat: number;
  lane: number;
  athleteId?: string;
  athleteName?: string;
  athleteTeam?: string;
  exhibition: boolean;
  status: "pending" | "official" | "dq" | "dns";
  officialTimeMs?: number;
  decidedAt?: number;
  decidedBy?: string;
  watchIds?: string[];
}
```

No orphan UUIDs to generate, no sync mapping between client strings and DB IDs, and INSERT INTO swims ... ON CONFLICT (event_id, heat, lane) DO UPDATE makes deck swaps and updates completely atomic.

User prompt: Okay, how about watches then?

Response: **Keep an explicit UUID / id on Watch. Do not drop it.**

Unlike Entry and Swim, a Watch is not a physical slot—it is an **event submission**. Attempting to make its primary key composite breaks down under real deck conditions.

Why a Composite Key Fails on Watch

To drop id on Watch, you would need a composite key like: PRIMARY KEY (event_id, heat, lane, slot) or PRIMARY KEY (event_id, heat, lane, user_id)

Both fail in common pool deck scenarios:

1. The Multi-Timer & Coach Overlap Problem:

As established earlier, you have volunteer timers (Slots 1–3), table admins typing paper backups, and coaches clicking splits across multiple lanes. If you key by (event_id, heat, lane, slot), where does Coach Mike's split go? If you key by user_id, anonymous volunteer devices or swapped phones collision-lock.

2. Network Retries & True Idempotency:

When a volunteer taps "Submit & Advance" as their phone passes behind the locker room, the HTTP POST or cookie queue might retry two or three times. If each submission has a unique id generated by the client at the exact moment of the button press, it acts as an **idempotency key**:

```
INSERT OR IGNORE INTO watches (id, event_id, heat, lane, ...) VALUES (?, ?, ?, ?, ...);
```

Retried packets silently drop with zero risk of overwriting or updating existing records.

3. Discards & Audit Trail:

If Timer 2 accidentally triggers their watch late and submits a 42.10 while Timers 1 and 3 submitted 28.40, the admin needs to flag or discard that specific watch submission. Having a discrete id (DELETE FROM watches WHERE id = ?) is clean and avoids deleting or affecting other watches in that lane.

The Clean Separation of Concerns

- **Swim and Entry are slots / entities:** They represent physical coordinates in the pool or declared positions on paper. Their coordinates (event_id, athlete_id) and (event_id, heat, lane) are their identities.
- **Watch is a submission / fact:** It represents a specific device recording an observation at a specific millisecond. It needs an immutable record ID.

Keep id: string (UUID) as the primary key on Watch.

User prompt: But the same benefits apply to watch, especially the one about table scans, to an even bigger extent since watches will be the most numerous thing in the system. A natural compound key would be event ID, heat, lane, userId, slot (knowing this now changes the data model we decided on for watch, re-introducing slot). Updates on the client and the server when DO messages are sent/received become an order one operation and remain idempotent very easily

Response: If you re-introduce slot as a designated integer alongside userId to form the composite key:

PRIMARY KEY (event_id, heat, lane, user_id, slot)

this setup creates a specific concurrency trap when applied to deck reality.

While table scans are avoided (and SQLite's B-tree index on composite keys is fast), look closely at what that specific composite tuple means in practice:

The Identity Hazard: What Does That Tuple Actually Represent?

A primary key must guarantee **uniqueness of intent**.

If the primary key is (event_id, heat, lane, user_id, slot):

1. Anonymous / Volunteer Device Handoffs:

Volunteer timers on deck are often unauthenticated parents handed a link (/timer/lane/3). If user_id is an anonymous device ID (or null/blank), what happens when a volunteer's battery dies mid-meet and they hand the lane over to a spouse, or borrow another phone?

- If Device A was Slot 1, and Device B takes over Slot 1, you now have **two distinct primary keys** for the same physical watch slot:
 - (ev_1, h_1, l_3, dev_A, 1)
 - (ev_1, h_1, l_3, dev_B, 1)

- The database now thinks Lane 3 has two different "Slot 1" watches.

2. Double-Click / Re-Submit Race Conditions:

If a volunteer double-clicks or adjusts a time before hitting submit, does the update target the exact same row? Yes, *if* `user_id` is identical. But if `user_id` changes (e.g. an admin takes over the lane on their own iPad), it creates an orphan instead of an overwrite.

The Two Valid Composite Models

If you want the compound key benefits for Watch (direct $O(1)$ lookups, zero table scans, and native UPSERT without surrogate UUIDs), you have to choose **which variable defines uniqueness**: the *Physical Slot* or the *Device*.

Option A: Keyed by Physical Slot Role (Recommended for Meet Ops)

If your primary goal is to collect up to 3 official watches, 1 admin override, and coach splits, let the **source slot** be the final discriminator, and keep `userId` as an informational metadata column:

```
PRIMARY KEY (event_id, heat, lane, source_slot)
```

Where `source_slot` is:

- 1, 2, 3 (Official Lane Timers)
- 'admin' (Table manual override)
- 'coach:mike' (Coach splits)

Why this is cleaner:

- **True Idempotency**: If volunteer phone A dies and volunteer phone B submits for Slot 1, `INSERT ... ON CONFLICT (event_id, heat, lane, source_slot) DO UPDATE` smoothly replaces Slot 1. You never get ghost duplicate watches for the same block.
- **Client Key**:

```
export type WatchKey = `${string}:h${number}:l${number}:s${string | number}`;
// meet.watches["ev_101:h2:l4:s1"]
```

It's an exact $O(1)$ slot lookup on the client.

Option B: Keyed by Device Submission (If Anyone Can Time Any Lane)

If you do not want slots, and any coach or timer can record times freely:

```
PRIMARY KEY (event_id, heat, lane, device_id)
```

Why this works:

- A single device can only submit one time per physical swim. If they retry, it naturally updates their recorded time.
- **Client Key**:

```
export type WatchKey = `${string}:h${number}:l${number}:d${string}`;
// meet.watches["ev_101:h2:l4:d_dev123"]
```

The Verdict on Composite Keys for Watches

Dropping the surrogate UUID on Watch works well, provided **you do not include both `user_id` and `slot` in the compound key**.

Pick one:

- If the app assigns volunteers to roles ("You are Timer 1 on Lane 4"):

```
PRIMARY KEY (event_id, heat, lane, slot)
```

(with `user_id` stored as a regular column for audit tracking).

- If timing is completely ad-hoc and unassigned:

```
PRIMARY KEY (event_id, heat, lane, device_id)
```

Option A is generally preferable for high school dual meets because it guarantees you never have four "official" watches competing for three lane positions when volunteers swap phones.

User prompt: Ah. I see there is confusion because I forget to share that every time is assigned a unique identifier when they sign up to time (they scan a QR code and it's randomly assigned based on their device). So, every watch submission would have a unique identifier for that user/device.

For logged in users, this is their user ID... For timers not explicitly required to login, it's their device ID. So the compound key built from ID, heat, lane, user/device ID, slot (where slot would normally just be "0" unless a timer is submitting for others in clipboard mode) actually works and has all the benefits we discussed without the drawbacks... I'm a fan of sticking to natural unique keys instead of always assigning IDs

Response: That context ties the whole picture together.

Because every person timing on deck has a deterministic `user_or_device_id` established at QR scan / session start, and `slot` handles the multi-watch clipboard scenario (e.g., a head timer or parent typing in Watch 1, 2, and 3 from paper), the composite key:

PRIMARY KEY (event_id, heat, lane, user_id, slot)

is a legitimate **natural composite key**. It represents: *"This specific device's recording for this specific slot in this specific race."*

Why This Works Cleanly

1. Eliminates the Device Swap Hazard:

Because the key explicitly includes `user_id`, if Phone A records a time and Phone B later steps in to record a time, they are distinctly identifiable submissions rather than blind, accidental overwrites of an anonymous slot.

2. Handles the Clipboard Mode Exception:

When a timer is running standard single-watch mode, `slot = 0`. When a chief timer or table admin uses the clipboard/bulk entry view to transcribe two backup watches from a paper lane card, they submit with `slot = 1` and `slot = 2` under their own ID. Both persist without colliding.

3. Idempotency on Flaky Connections:

If a volunteer's phone resubmits a queued watch packet three times as Wi-Fi drops and reconnects, SQLite's `INSERT ... ON CONFLICT (event_id, heat, lane, user_id, slot) DO UPDATE` absorbs the retries seamlessly.

The Clean Client Representation

Now your client key, database schema, and socket messages are 100% aligned with zero surrogate IDs anywhere in the timing chain:

```
// app/lib/meetCache.ts

export type WatchKey = `${string}:h${number}:l${number}:u${string}:s${number}`;

export interface WatchSlotKey {
  eventId: string;
  heat: number;
  lane: number;
  userId: string; // auth user ID or anonymous device ID from QR
  slot: number;   // 0 for direct timing, 1/2/3 for multi-watch clipboard transcription
}

export function toWatchKey(k: WatchSlotKey): WatchKey {
  return `${k.eventId}:h${k.heat}:l${k.lane}:u${k.userId}:s${k.slot}`;
}

export function parseWatchKey(key: WatchKey): WatchSlotKey {
  const [eventId, h, l, u, s] = key.split(":");
  return {
    eventId,
    heat: Number(h.replace("h", "")),
    lane: Number(l.replace("l", "")),
    userId: u.replace("u", ""),
    slot: Number(s.replace("s", "")),
  };
}

export interface Watch {
  eventId: string;
  heat: number;
  lane: number;
  userId: string;
  slot: number;
  submittedBy: string; // Display name (e.g. "Mike R.", "Lane 3 Timer")
  role: "timer" | "chief_timer" | "admin" | "coach";
  timeMs?: number;
  startedAt?: number;
  stoppedAt?: number;
  submittedAt: number;
}
```

The Full Stack Architecture at a Glance

Dropping surrogate UUIDs across the board leaves your core entities completely normalized by their natural physical and operational identities:

Entity	Composite Primary Key	Client Manifest Key	What It Identifies
Event	id	event.id	The swim event definition
Entry	(event_id, athlete_id)	\${eventId}:\${athleteId}	Athlete seed declaration
Swim	(event_id, heat, lane)	\${eventId}:h\${heat}:l\${lane}	The physical slot in the water
Watch	(event_id, heat, lane, user_id, slot)	\${eventId}:h\${heat}:l\${lane}:u\${userId}:s\${slot}	A specific device's timing observation

This keeps your client memory footprint lean, ensures every WebSocket patch is an instant $O(1)$ write, and completely matches the physical reality of what is happening on deck.

User prompt: ok - back to an architectural question... the app has two modes of functioning - public facing portal for swim data (meets, athletes, teams) and a swim meet manager. i've current got to two databases - a D1 database (longer term, used by the remix public site to store all data) and the SQL db used by my durable object (read/written to while a meeting is running)... the DO SQL db only has a subset of the data (that required for the one meet it's running), although all of the data has the same schema. data in the DO is hydrated when it starts up (if it doesn't exist already) and when a meet is done, the DO data is dumped back into the identical tables in D1. however - i'm now rethinking whether this is necessary... initially it seemed you were just supposed to use DO sql's data store during websocket messages...but i could just as easily use D1 to persist, right? and, that would avoid the duplication? does this split approach still make sense? are there cost or performance considerations for maintaining everything in one DB?

Response: Stick with the **Durable Object SQLite + D1 split approach**. Do not route active deck writes directly to D1.

While bypassing the DO's embedded SQL to write straight to D1 sounds like it simplifies synchronization, it reintroduces the exact latency, concurrency, and reliability risks you are trying to protect against on a live pool deck.

1. Latency & Touch Responsiveness: Microseconds vs. Milliseconds

The DO's embedded SQLite runs **co-located in the same memory and disk space** as your Durable Object instance.

- A write to DO SQLite: < 0.5 ms (in-process SQLite transaction on local NVMe storage).
- A write to D1 from a DO or Worker: 15 ms to 80+ ms (network hop across the Cloudflare internal spine to the primary D1 instance, plus distributed consensus/coordination).

When a heat finishes and 24 watches hit the system in a 2-second burst, running 24 writes against in-process SQLite is negligible. Running 24 individual network calls out to D1 introduces network jitter, pool connection contention, and potential queue lag right as the official needs to declare a race official.

2. The Concurrency & Lock Bottleneck in D1

D1 is built on SQLite replicated over the edge, but all writes must serialize through a **single primary database node**:

- If 8 lane timers, 2 coaches, and 1 admin are actively submitting, updating, and re-reading D1 simultaneously, every write must queue for the single-writer lock on that D1 database.
- If a public user on the spectator site is running a heavy query (e.g., "Show team point standings and historical splits across the last 3 meets"), that read or transaction shares resources with your active deck timers.
- With the DO architecture, your live meet has its **own isolated SQLite engine and CPU sandbox**. Spectators hammering the public portal cannot degrade the timing engine on deck.

3. Costs: Row Writes Add Up Fast on D1

Cloudflare pricing makes this trade-off stark:

- **D1 Pricing:** Billed on **Rows Read** and **Rows Written**.
 - A heat finishes: 24 watch inserts, status updates, swim official time updates, re-querying entries.
 - Over a 48-event meet with preliminary scratches, reseeds, and split submissions, you can easily log tens of thousands of D1 row writes per meet.
- **Durable Object SQLite Pricing:** Billed on **DO Duration / Memory (GB-s)** and **DO Request Units**.
 - DO SQLite queries and row writes **do not incur D1 row-write charges**.

- You can perform 10,000 in-memory SQLite transactions inside a running DO during a 2-hour meet for virtually zero extra storage cost beyond standard DO compute time.

4. Offline Resilience and the Single Source of Truth

When a meet is live, the **Durable Object is the single source of truth**, not D1.

- The DO is already coordinating the WebSockets.
- The DO maintains the atomic state machine for "Is this heat locked?", "Has Lane 4 touched?", and "Are times unanimous?".
- If the DO writes to its own local SQLite, that write is guaranteed to be synchronous and consistent with its broadcast messages.
- If the DO had to write to external D1, you would have to handle two-phase rollbacks: what happens if the D1 HTTP call times out or fails, but the DO already broadcasted the touch to the admin grid?

The Clean Way to Handle the Split (Without Manual "Dumping")

The friction you are feeling isn't the split itself; it's the feeling of having to write an awkward, error-prone "bulk dump" script at the end of the meet.

You can streamline this using a **Transactional Meet Finalization** or a **Write-Behind Flush**:

Pattern A: End-of-Meet Archival (Simplest & Most Reliable)

1. **During Meet:** The DO owns the meet. All mutations (swims, watches, deck scratches) happen in DO SQLite. D1 is not touched for live timing data.
2. **Spectator Live Views:** Public spectators watching the live meet read from the DO or subscribe to its public read-only WebSocket channel.
3. **When Admin Taps "Finalize Meet":**
 - The DO serializes its finalized swims and watches tables.
 - A single batch transaction inserts them into D1:

```
// Inside DO finalize action
await env.D1.batch([
  ...swims.map(s => insertSwimD1Statement(s)),
  ...watches.map(w => insertWatchD1Statement(w)),
  env.D1.prepare("UPDATE meets SET status = 'completed' WHERE id = ?").bind(meetId)
]);
```

- D1 is now fully populated for historical archiving, leaderboards, and seasonal analytics.
- The DO can shut down or destroy its temporary tables.

Pattern B: Fire-and-Forget Write-Behind (If D1 Must Stay Semi-Current)

If the public site *must* read finished heats directly from D1 while the meet is still going:

- When an admin marks a heat "Official", the DO commits it to local SQLite immediately.
- It then asynchronously queues a batch write to D1 using Cloudflare's `ctx.waitUntil(...)`:

```
// Synchronous, instant local commit
this.db.exec("UPDATE swims SET status = 'official' WHERE ...");

// Non-blocking background flush to D1
this.ctx.waitUntil(
  this.env.D1.batch([/* batch insert finished heat into D1 */])
);
```

The deck never waits on D1, but D1 stays within a few seconds of the live pool.

The Verdict

Keep the two databases:

- **Durable Object SQLite:** The high-performance, isolated, ephemeral deck engine (optimized for concurrency, sub-millisecond writes, and WebSocket coordination).
- **D1:** The relational warehouse (optimized for global edge queries, multi-meet seasonal reporting, athlete profiles, and team rosters).

This separation is one of Cloudflare's biggest architectural superpowers—it gives you dedicated micro-databases per live meet without running servers.

User prompt: Well...worth having a DO for every meet and just keeping all meet-specific stuff there? The d1 db would essentially just be an index of the available DO/meet-specific dbs? Ie my meets list/index page would call d1 and get super basic info for each meet (I'd, name, date, status)... browsing to any meets detail page would involve contacting the DO's data store (ie entries, meet details, results plus the obvious live meet stuff around watches, etc)?

Response: This pattern—often called the "Actor-as-a-Database" or Database-per-Tenant model—is one of Cloudflare's most powerful architectures, but when you push *all* historical results and detail views into individual DOs, you run into real trade-offs between write isolation and analytical querying.

The Reality of "DO-as-the-Only-Store"

What Becomes Incredibly Clean:

1. Zero Sync Overhead:

You eliminate the "dump back to D1" pipeline completely. When a heat finishes or an official signs off on results, the data is already in its permanent home.

2. Infinite Scaling & Complete Tenant Isolation:

If 20 high schools are running championships on the same Saturday morning, each meet runs on its own isolated SQLite engine, disk space, and CPU budget. No single busy meet or locked table can slow down another.

3. Natural Lifecycle:

When a meet is idle, the DO hibernates to cold storage on Cloudflare's disk for fractions of a penny. When someone views /meets/123/results, Cloudflare spins up the DO in 5–10ms, reads its local SQLite, returns the JSON, and shuts it down.

The Big Catch: Cross-Meet Aggregation & Athlete Profiles

The DO-only architecture works smoothly as long as you only look at **one meet at a time**.

It breaks down the moment your public portal needs to query **across meets**:

- "Show Marcus Vance's 50 Free times across the entire season to see if he qualified for State."
- "What is the school record for the Boys 100 Breaststroke across all meets in 2026?"
- "Show team scoring and standings over the four-meet series."

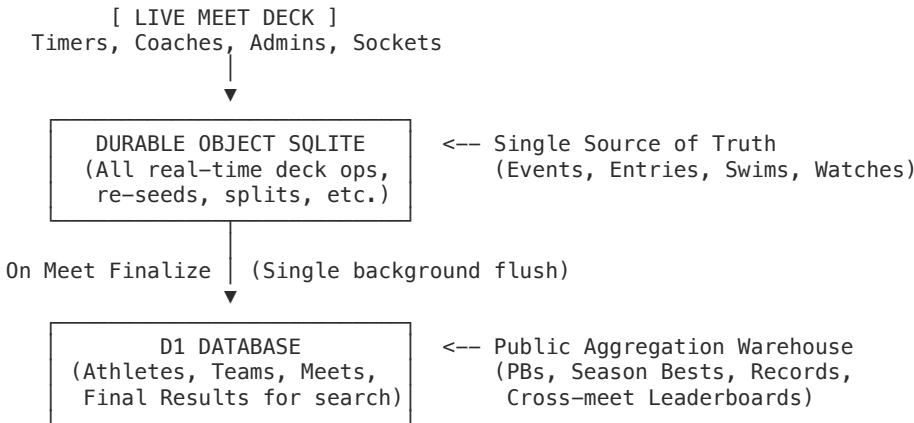
If meet data lives solely inside individual DOs, computing an athlete's personal best (PB) requires:

1. Querying D1 to find all meet IDs for the season.
2. Waking up 8 separate Durable Objects over network RPC calls.
3. Querying SQLite inside each DO.
4. Manually stitching, sorting, and aggregating the results in a Worker.

That turns a simple SQL `SELECT MIN(time_ms) FROM swims WHERE athlete_id = ?` into an expensive fan-out operation that will blow past Worker CPU limits and feel sluggish.

The Hybrid Sweet Spot: DO as Authoritative Primary, D1 as Read-Optimized Warehouse

Instead of treating D1 as an operational store that you constantly sync with, treat D1 as a **read warehouse** populated via an asynchronous event stream:



How the Views Divide Cleanly:

1. The Live Meet & Meet Detail Page (/meets/:id/...):

Talks **directly to the DO**.

- It loads the full MeetManifest from the DO's local SQLite.
- Timers, coaches, and spectators viewing that specific meet get instant, authoritative data straight from the DO.

2. The Public Portal & Season Analytics (/athletes/:id, /records, /rankings):

Talks **directly to D1**.

- D1 contains the global index of athletes, teams, and finished meet results.
- Running `SELECT * FROM results WHERE athlete_id = ? ORDER BY time_ms ASC` takes 2ms at the edge without waking up a single DO.

Summary Checklist: Should You Put It All in the DO?

- **If your app will NEVER need cross-meet reporting** (it is purely a standalone meet scoring tool that produces a PDF/CSV at the end):

Yes, keep everything in the DO. D1 just holds a list of { `meetId`, `name`, `date` }, and you never have to sync to D1 at all.

- **If your public portal features athlete bios, personal bests, team leaderboards, or state cut trackers:**

Keep the DO as the **authoritative operational store** for the meet, but run a single async export of finalized swims into D1 when the meet concludes. That gives you the best of both worlds: zero lock contention during the meet, and instant relational queries across the whole season.

User prompt: Love this... i'm thinking D1 would still have a basic meet table and a results table...that's where approved results go once a meet completes and the admin "closes" it and it's flushed to D1... I think that would require conditionally pulling meet details from DO vs d1 depending on if the meet is marked as complete or not?

Response: Yes, that routing logic fits the lifecycle cleanly.

Once a meet is closed and flushed to D1, keeping the DO around as the primary read target for public traffic is unnecessary. Shifting historical reads to D1 keeps DO storage light, avoids cold-start wakeups on archived meets, and lets Cloudflare's read-replicated edge cache handle public traffic.

How to Model the Route & Data Switch

In your Remix loader for the meet detail/results page (/meets/:id), you can implement this branching in a few lines. Because meets in D1 already tracks status, you check D1 first—or use D1's record as the routing gate.

```
// app/routes/meets.$id._index.tsx
import { json, type LoaderFunctionArgs } from "@remix-run/cloudflare";

export async function loader({ params, context }: LoaderFunctionArgs) {
  const meetId = params.id!;
  const { env } = context;

  // 1. Check basic meet metadata from D1 (fast, edge-cached)
  const meetSummary = await env.D1.prepare(
    "SELECT id, name, date, status FROM meets WHERE id = ?"
  )
    .bind(meetId)
    .first<{ id: string; name: string; date: string; status: "draft" | "live" | "completed" }>();

  if (!meetSummary) {
    throw new Response("Meet Not Found", { status: 404 });
  }

  // 2. If COMPLETED: Pull finalized results directly from D1 warehouse
  if (meetSummary.status === "completed") {
    const results = await env.D1.prepare(
      `SELECT r.*, a.first_name, a.last_name, a.team_name, e.event_number, e.name as event_name
      FROM results r
      JOIN athletes a ON r.athlete_id = a.id
      JOIN events e ON r.event_id = e.id
      WHERE r.meet_id = ?
      ORDER BY e.event_number ASC, r.place ASC`
    )
      .bind(meetId)
      .all();

    return json({
```

```

    source: "archive",
    meet: meetSummary,
    results: results.results,
  });
}

// 3. If DRAFT or LIVE: Delegate entirely to the authoritative Durable Object
const doId = env.MEET_D0.idFromName(meetId);
const stub = env.MEET_D0.get(doId);

// Directly grab the in-memory/D0-SQLite manifest
const manifest = await stub.getMeetManifest();

return json({
  source: "live",
  meet: manifest,
});
}

```

Why This Split Is Operationally Sound

1. Cold Meets Stay Cold:

When a parent looks up results from a meet that took place three weeks ago, Cloudflare serves it straight from D1 (or Edge Cache). The Durable Object for that meet remains completely hibernated on disk—zero CPU billing, zero memory allocation.

2. D1 results Table Stays Lean and Analytical:

Notice that D1 does **not** need raw watches or unseeded lane shells. D1 only needs the final, approved business record:

```

CREATE TABLE results (
  meet_id TEXT NOT NULL,
  event_id TEXT NOT NULL,
  athlete_id TEXT NOT NULL,
  heat INTEGER,
  lane INTEGER,
  time_ms INTEGER NOT NULL,
  place INTEGER,
  points REAL,
  status TEXT NOT NULL, -- 'official', 'dq', 'exhibition'
  PRIMARY KEY (meet_id, event_id, athlete_id)
);

```

All the deck chatter (watches 1–3, starter timestamps, scratch churn) stays sealed inside the DO archive, keeping your analytical D1 queries fast.

3. Re-Opening a Closed Meet Is Still Possible:

If an official discovers a DQ error after the meet was finalized and needs to amend results, an admin action can simply update D1's status back to live, wake up the DO, adjust the swim, and re-trigger the finalize flush.

The Meet Lifecycle Pipeline

```

[ Meet Setup / Draft ]  → Managed in D0 (seeding, scratches, entries)
                        |
[ Meet In-Progress ]   → Managed in D0 (live WS deck chatter, watches, heat status)
                        |
[ Admin Taps "Close" ] → D0 executes:
                        1. Calculates final places & team scores
                        2. Batch inserts into D1 `results` table
                        3. Updates D1 `meets` status = 'completed'
                        |
[ Post-Meet Archival ] → All traffic routes to D1:
                        • Meet result view (/meets/:id)
                        • Athlete PR / Season history (/athletes/:id)
                        • Team scoreboards & rankings

```

This keeps the live deck operations isolated from global analytics without needing complex real-time synchronization between the two databases.

User prompt: In this model, if I'm not mistaken, each turbo object has its own sequel beta base meaning I could actually drop the meat ID columns from all of my live operational entities two right

Response: Yes, absolutely. You can completely drop `meet_id` from every operational table inside the Durable Object.

Every Durable Object instance has its own private, isolated embedded SQLite database. The database file itself is bound directly to that specific DO identity:

```
// The DO instance IS the meet
const doId = env.MEET_DO.idFromName(meetId);
```

Because `meet_123` can never physically access or co-mingle data with `meet_456`'s SQLite database, having a `meet_id` column on events, entries, swims, or watches inside the DO provides zero relational, partitioning, or filtering value.

What Your DO SQLite Schema Becomes

Dropping `meet_id` strips away the last bit of redundant database overhead:

-- Inside the DO's private SQLite:

```
CREATE TABLE events (
  id TEXT PRIMARY KEY,
  position INTEGER NOT NULL,
  event_number INTEGER NOT NULL,
  distance INTEGER NOT NULL,
  stroke TEXT NOT NULL,
  gender TEXT NOT NULL,
  name TEXT,
  total_heats INTEGER NOT NULL DEFAULT 1
);

CREATE TABLE entries (
  event_id TEXT NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  athlete_id TEXT NOT NULL,
  seed_time_ms INTEGER,
  is_exhibition INTEGER NOT NULL DEFAULT 0,
  submitted_by_team_id TEXT NOT NULL,
  PRIMARY KEY (event_id, athlete_id)
);

CREATE TABLE swims (
  event_id TEXT NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  heat INTEGER NOT NULL,
  lane INTEGER NOT NULL,
  athlete_id TEXT,
  athlete_name TEXT,
  athlete_team TEXT,
  exhibition INTEGER NOT NULL DEFAULT 0,
  status TEXT NOT NULL DEFAULT 'pending',
  official_time_ms INTEGER,
  decided_at INTEGER,
  decided_by TEXT,
  PRIMARY KEY (event_id, heat, lane)
);

CREATE TABLE watches (
  event_id TEXT NOT NULL,
  heat INTEGER NOT NULL,
  lane INTEGER NOT NULL,
  user_id TEXT NOT NULL,
  slot INTEGER NOT NULL DEFAULT 0,
  submitted_by TEXT NOT NULL,
  role TEXT NOT NULL DEFAULT 'timer',
  time_ms INTEGER,
  started_at INTEGER,
  stopped_at INTEGER,
  submitted_at INTEGER NOT NULL,
  PRIMARY KEY (event_id, heat, lane, user_id, slot),
  FOREIGN KEY (event_id, heat, lane) REFERENCES swims(event_id, heat, lane) ON DELETE CASCADE
);
```

Why This Is Superior Inside the DO

1. Zero Index Overhead:

In a multi-tenant DB (like standard D1), you would need composite indexes starting with `meet_id` on every table (`INDEX idx_swims_meet ON swims(meet_id, event_id)`). In the DO, the entire database *is* the index for that meet. Primary keys and queries are smaller and execute faster.

2. Clean Querying:

Every operational query inside the DO becomes simpler:

```
-- Instead of: SELECT * FROM swims WHERE meet_id = ? AND event_id = ? AND heat = ?;
SELECT * FROM swims WHERE event_id = ? AND heat = ?;
```

3. 100% Symmetrical with Client POCOs:

Earlier, we decided to drop `meetId` from client models (`Swim`, `Entry`, `Watch`) because the URL provided context. Now your client POCOs and your DO SQL rows match property-for-property with zero transformation mapping.

Where `meet_id` Remains

`meet_id` is only needed in **D1**:

- `meets`: The global list of meets (`id`, `name`, `date`, `status`).
- `results`: The final post-meet warehouse table where all meets live together (`meet_id`, `event_id`, `athlete_id`, `time_ms`, etc.).

When the meet concludes and the DO flushes approved results to D1, the DO simply injects `this.meetId` into the D1 batch insert:

```
// Inside DO finalize()
const swims = this.sql.exec("SELECT * FROM swims WHERE status = 'official'").toArray();

await this.env.D1.batch(
  swims.map(swim =>
    this.env.D1.prepare(
      `INSERT INTO results (meet_id, event_id, athlete_id, heat, lane, time_ms, status)
      VALUES (?, ?, ?, ?, ?, ?, ?)`
    ).bind(
      this.meetId, // Injected once at the boundary
      swim.event_id,
      swim.athlete_id,
      swim.heat,
      swim.lane,
      swim.official_time_ms,
      swim.status
    )
  )
);
```

Inside the DO, every table is strictly scoped to that meet by definition, so dropping `meet_id` is clean, lean, and correct.

Sources

1. [A Beginner's Guide to Cloudflare Durable Objects - OpenReplay Blog](#)

User prompt: I know we've been over this before and can't keep to a decision, but in light of simplifying our data models I'm wondering again if we need both the entry and the swim concert... If we added seed time to our swim model, it's exactly the same as entry. We would also modify our entries user interface and API, so that entries could never be deleted. Once there was an associated watch entry. Would Syl be allowed anytime... A receipt would only occur if no watches were submitted for that event the same way as today. What do you think?

Response: **Do not collapse Entry and Swim into a single entity.**

While collapsing them looks tempting on paper—especially when trying to strip down boilerplate—merging intent (`Entry`) with physical execution (`Swim`) breaks fundamental deck workflows the moment real-world friction hits.

Here is why keeping them separate is necessary for an offline-resilient meet manager:

1. Where Can an Unseeded Swimmer "Live"?

If `Swim` is your only table, its primary key is (`event_id`, `heat`, `lane`).

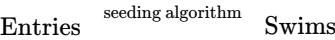
Before a race is seeded, where does an athlete exist?

- If Coach A submits 14 swimmers for the 50 Free on Tuesday, they don't have heat and lane assignments yet.
- If you collapse them into `Swim`, what are their heat and lane? NULL? 0?
- As soon as you have (`event_id`, `heat: null`, `lane: null`), your primary key (`event_id`, `heat`, `lane`) completely collapses. You can't have 14 nulls in a primary key. You'd be forced right back into generating artificial surrogate UUIDs just to hold unseeded roster entries.

2. The Reseed Problem (Destructive vs. Non-Destructive)

In high school meets, reseeding happens constantly: three kids don't show up on the bus, leaving empty lanes, so the coach or clerk consolidates Heat 1 and Heat 2 into a single heat.

- **With Separate Models:** Reseeding is a clean functional projection:



To reseed Event 4, you simply delete the un-swum `swims` for Event 4 and re-run the generator against the immutable `entries` table. The athletes, their original seed times, and their exhibition statuses are completely untouched.

- **With a Single Collapsed Model:** If an entry *is* a swim, "reseeding" means wiping or mutating the very rows that hold the athlete declarations. If you clear the heats and lanes to re-pack them, you risk deleting the declaration itself or corrupting who was entered if the operation is interrupted.

3. Scratches vs. Empty Lanes

What happens when an athlete scratches 10 minutes before the race?

- **Real Deck Reality:** Marcus scratches from Heat 2, Lane 4.
 - Marcus's declaration status becomes `scratched` (or is removed from entries).
 - Lane 4 in Heat 2 **still physically exists in the pool**.
 - The lane timers are standing behind Lane 4 with their stopwatches.
- If `Entry` and `Swim` are merged:
 - Do you delete the row? If so, Lane 4 disappears from the timer's clipboard and stopwatch screens. The timer app now thinks the pool only has 5 lanes.
 - Do you keep the row and null out `athlete_id`? Now you've lost the record of the entry, or you're managing complex nullable state machines on what should be a concrete pool slot.

4. Walk-Ups and Last-Second Lane Swaps

Coaches regularly make verbal swaps right at the blocks: *"Hey, put Sarah in Heat 1 Lane 2 instead of Heat 2 Lane 5."*

- When Sarah steps up, the lane timer or admin simply changes the `athlete_id` on that specific `Swim` slot (`event_id`, `heat`, `lane`).
- If `Swim` and `Entry` are merged, moving Sarah to Lane 2 either:
 1. Overwrites whoever was seeded in Lane 2.
 2. Violates unique constraints.
 3. Fails because Sarah already has a row in that event.

The Clean Mental Boundary

Think of the distinction not as duplicate data, but as **Declaration vs. Execution**:

Layer	Entity	Key	Meaning
Declaration	Entry	(<code>event_id</code> , <code>athlete_id</code>)	<i>Who is signed up to swim this race, and what is their seed time?</i>
Execution	Swim	(<code>event_id</code> , <code>heat</code> , <code>lane</code>)	<i>What physical body is in the water, and what time did the watches record?</i>

Dropping surrogate UUIDs made both models lean and elegant. Keeping them separate preserves the natural state lifecycle of a meet: entries exist before heats are built, survive reseeds, and allow physical lanes to remain open, swappable, and timeable regardless of deck scratches.

Sources

1. [Seeding Basics and Tips - Hy-Tek](#)